

UNIVERSIDADE FEDERAL DE PERNAMBUCO
CENTRO DE INFORMÁTICA
SISTEMAS DIGITAIS — 2026.2

Projeto da Primeira Unidade

Unidade Lógica e Aritmética de 5 bits com displays de 7 segmentos

Relatório de projeto — placa Terasic DE2-115 (Cyclone IV E EP4CE115F29C7)

André Lopes Aragão (ala8)

Daniel Bezerra de Moraes (dbm5)

Enzo Priori (ep)

Pedro Augusto Bandeira Paz de Andrade (pabpa)

Professor: Abel Guilhermino

Repositório: github.com/ep-creator/Projeto1SD

Recife, outubro de 2026

Sumário

1 Introdução.....	4
2 Visão geral do projeto.....	5
2.1 Operações.....	5
2.2 Entradas e saídas.....	5
2.3 Diagrama em blocos.....	5
2.4 Módulos desenvolvidos.....	6
3 Fundamentos e decisões de projeto.....	8
3.1 Sinal-magnitude e complemento a 2.....	8
3.2 Soma e subtração.....	8
3.3 Complemento a 2 de B.....	9
3.4 Comparações em sinal-magnitude.....	9
3.5 Displays de 7 segmentos.....	9
3.6 Decisões de projeto.....	9
4 Projeto dos módulos.....	11
4.1 Blocos comuns.....	11
4.1.1 mux2x1.....	11
4.1.2 mux4x1.....	12
4.2 Complemento a 2.....	14
4.2.1 inversor.....	14
4.2.2 inversor5.....	15
4.2.3 comp2.....	16
4.3 Soma e subtração.....	18
4.3.1 somador_completo.....	18
4.3.2 somador6.....	20
4.3.3 c2_para_sm.....	21
4.3.4 somador_subtrator.....	23
4.4 Operações lógicas.....	25
4.4.1 op_and.....	25

4.4.2 op_xor.....	26
4.5 Comparações.....	28
4.5.1 comp_mag.....	28
4.5.2 comp_maior.....	29
4.5.3 comparador_igual.....	31
4.5.4 decodificador_comparadores.....	32
4.5.5 mux_comparadores.....	34
4.6 Seleção da saída.....	35
4.6.1 decodificador_saida.....	35
4.6.2 mux_saida.....	37
4.7 Displays.....	38
4.7.1 decod7seg_ab_dezena.....	38
4.7.2 decod7seg_ab_unidade.....	41
4.7.3 decod7seg_f_dezena.....	44
4.7.4 decod7seg_f_unidade.....	47
4.7.5 decod7seg_f.....	51
4.7.6 apaga_display.....	52
5 Sistema completo.....	54
5.1 ULA.....	54
5.2 Toplevel.....	56
5.3 Pinagem.....	57
5.4 Teste na placa.....	58
6 Conclusão.....	60

1 Introdução

Este relatório descreve o projeto de uma Unidade Lógica e Aritmética (ULA) de 5 bits, acoplada a decodificadores para displays de 7 segmentos, desenvolvida para a placa de prototipação Terasic DE2-115. A ULA recebe dois operandos em sinal-magnitude e executa oito operações: soma, subtração, complemento a 2, três comparações (igual, maior e menor), AND e XOR bit a bit.

Como pede o enunciado, todo o projeto foi feito **somente com portas lógicas**, em esquemáticos (.bdf) no Quartus Prime Lite, sem HDL e sem megafunções. O sistema foi dividido em blocos pequenos, cada um projetado a partir da sua tabela verdade, simulado isoladamente e reutilizado nos blocos maiores. Os blocos ficam numa biblioteca única (lib/) do repositório do grupo, e cada um tem um projeto de teste com tabela verdade e simulação em testes/<bloco>/.

O relatório segue os itens do enunciado: a seção 2 apresenta a visão geral do sistema em blocos; a seção 3 resume os conceitos usados (sinal-magnitude, complemento a 2, comparação e displays); a seção 4 traz, para cada módulo, a tabela verdade, as equações e mapas de Karnaugh, o circuito e a simulação; a seção 5 mostra o sistema completo conectado, a pinagem e os testes na placa; a seção 6 conclui.

2 Visão geral do projeto

2.1 Operações

A ULA recebe dois números de 5 bits, A e B, e um seletor de 3 bits, S. Conforme S, ela calcula uma das operações da tabela abaixo e mostra o resultado em LEDs e displays.

Tabela 1 – operações

$S_2 S_1 S_0$	Operação	Resultado aparece em
000	$F = A + B$	LEDs verdes (F) e displays HEX2, HEX1 e HEX0
001	$F = A - B$	LEDs verdes (F) e displays HEX2, HEX1 e HEX0
010	$F = \text{complemento a 2 de B}$	LEDs verdes (F)
011	$A = B ?$	LED de STATUS
100	$A > B ?$	LED de STATUS
101	$A < B ?$	LED de STATUS
110	$F = A \text{ AND } B \text{ (bit a bit)}$	LEDs verdes (F)
111	$F = A \text{ XOR } B \text{ (bit a bit)}$	LEDs verdes (F)

2.2 Entradas e saídas

- **A e B:** 5 bits em sinal-magnitude, [sinal, M3, M2, M1, M0]. O bit de sinal é 1 para negativo e 0 para positivo; a magnitude vai de 0 a 15. Quem usa a placa não precisa saber complemento a 2: toda conversão é feita dentro da ULA.
- **S:** 3 bits, seleciona a operação.
- **F:** 6 bits em sinal-magnitude, [sinal, F4, F3, F2, F1, F0]. O resultado da soma e da subtração vai de -30 a +30.
- **STATUS:** 1 bit (LED), verdadeiro ou falso nas comparações.
- **Displays:** |A|, |B| e |F| em dezena e unidade; o sinal de F aparece como “-” no HEX2. Os displays de F só acendem na soma e na subtração.
- **LEDs:** F nos LEDs verdes, as chaves replicadas nos LEDs vermelhos (com os sinais de A e B em LEDR17 e LEDR12).

2.3 Diagrama em blocos

A Figura abaixo mostra o sistema completo: as chaves da placa alimentam a ula e os decodificadores de |A| e |B|; a saída F vai para os LEDs verdes e, passando pelo decodificador de F e pelo bloco que apaga os displays, para HEX1 e HEX0.

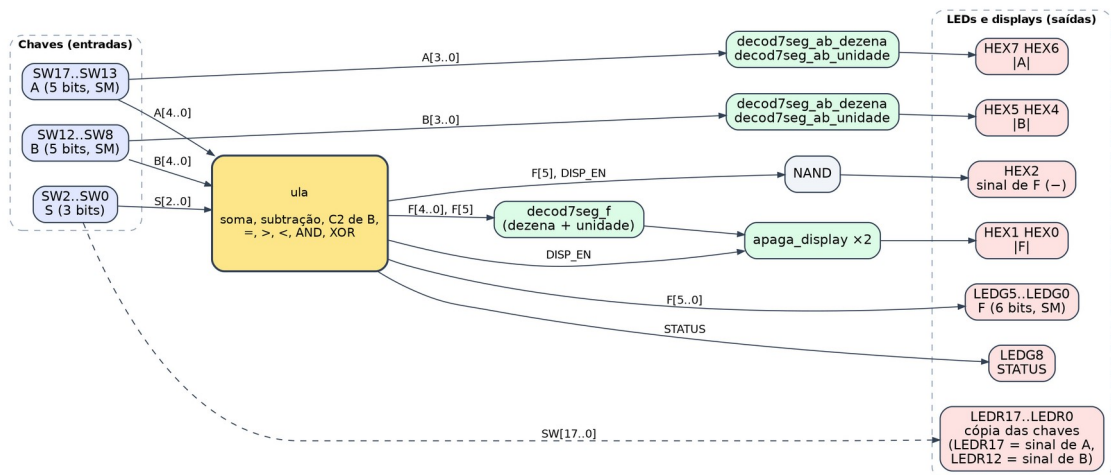


Figura 1 – Visão geral do sistema em blocos

Internamente, a u_{la} calcula todos os resultados em paralelo e escolhe um deles com multiplexadores controlados por S:

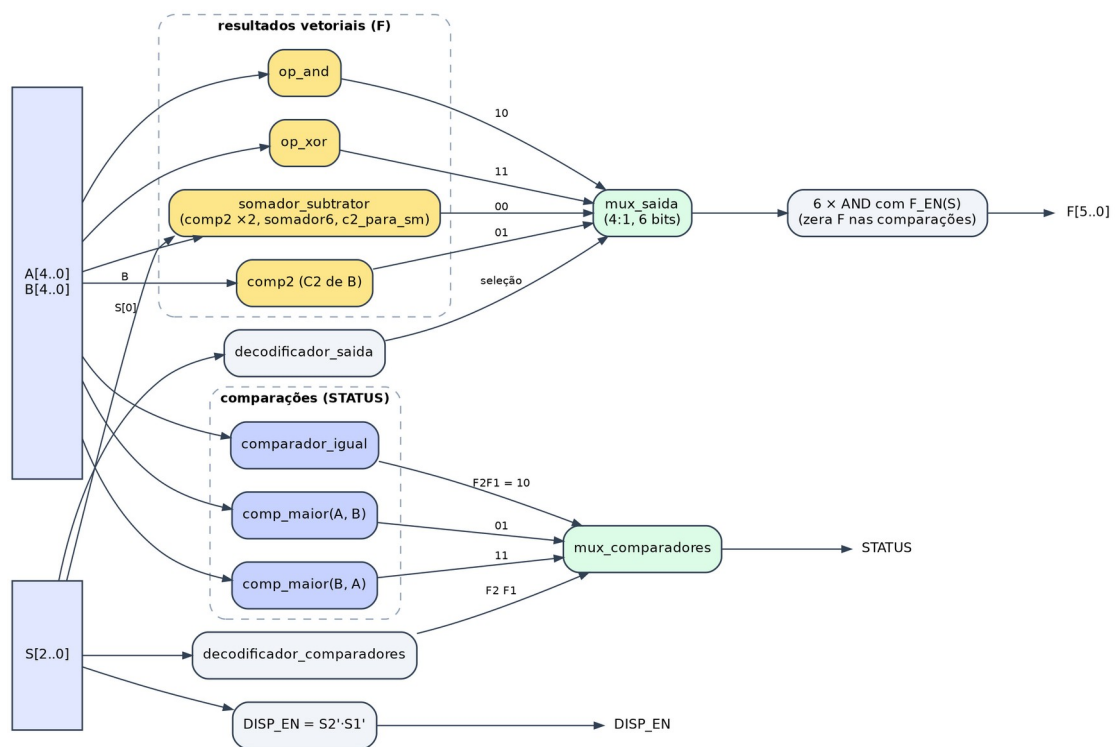


Figura 2 – Organização interna da ULA

2.4 Módulos desenvolvidos

Tabela 2 – módulos desenvolvidos

Grupo	Módulo	Função
Comuns	mux2x1, mux4x1	multiplexadores de 1 bit (2:1 e 4:1)
Complemento a 2	inversor, inversor5	complemento a 2 de 4 e de 5 bits
	comp2	sinal-magnitude → complemento a 2 (-0 vira

Grupo	Módulo	Função
		+0)
Soma e subtração	somador_completo	soma de 3 bits (célula do somador)
	somador6	soma em complemento a 2 de 6 bits (-30 a +30)
	c2_para_sm	complemento a 2 → sinal-magnitude
	somador_subtrator	$A \pm B$ em sinal-magnitude (operações 000 e 001)
Lógica	op_and, op_xor	AND e XOR bit a bit (110 e 111)
Comparações	comp_mag	$ A > B $ (magnitudes de 4 bits)
	comp_maior	$A > B$ em sinal-magnitude (100; trocando A e B, 101)
	comparador_igual	$A = B$, com +0 = -0 (011)
	decodificador_comparadores, mux_comparadores	escolhem a comparação que vai para o STATUS
Seleção	decodificador_saida, mux_saida	escolhem o resultado que vai para F
Displays	decod7seg_ab_dezena, decod7seg_ab_unidade	dezena e unidade de $ A $ e $ B $ (0 a 15)
	decod7seg_f_dezena, decod7seg_f_unidade, decod7seg_f	dezena e unidade de $ F $ (0 a 30)
	apaga_display	apaga os displays de F fora da soma e da subtração
Sistema	ula	liga todos os blocos e gera F, STATUS e DISP_EN
	toplevel	liga a ula e os decodificadores aos pinos da placa

A hierarquia de instâncias é:

```

toplevel
├── ula
│   ├── somador_subtrator — 000 / 001
│   │   ├── comp2 x2 — inversor, mux2x1 x4
│   │   ├── XOR (inverte o sinal de B na subtração)
│   │   ├── somador6 — somador_completo x6
│   │   └── c2_para_sm — inversor5, mux2x1 x5
│   ├── comp2 — 010
│   ├── op_and, op_xor — 110 / 111
│   ├── decodificador_saida + mux_saida (mux4x1 x6)
│   ├── comparador_igual — 011
│   ├── comp_maior x2 (comp_mag x2 cada) — 100 / 101
│   └── decodificador_comparadores + mux_comparadores
├── decod7seg_ab_dezena x2, decod7seg_ab_unidade x2
├── decod7seg_f (decod7seg_f_dezena + decod7seg_f_unidade)
├── apaga_display x2
└── NAND (sinal de F no HEX2)

```

3 Fundamentos e decisões de projeto

3.1 Sinal-magnitude e complemento a 2

Em **sinal-magnitude (SM)**, o bit mais alto é o sinal e os outros são o valor absoluto: $10101 = -5$. É o formato pedido para as entradas e para F, porque é fácil de ler, mas é ruim para fazer contas: somar -5 com $+3$ exige comparar as magnitudes e subtrair a menor da maior.

Em **complemento a 2 (C2)**, um único somador binário faz soma e subtração de números com sinal. O C2 de um número é obtido invertendo todos os bits e somando 1. Um atalho equivalente, usado nos blocos `inversor` e `inversor5`, é copiar os bits da direita até o primeiro 1 (inclusive) e inverter todos os outros; por isso cada bit de saída é o bit de entrada XOR “existe algum 1 abaixo dele”.

Por isso a ULA trabalha assim nas contas: **SM** $\rightarrow$ **C2** $\rightarrow$ **soma** $\rightarrow$ **C2** $\rightarrow$ **SM**. A entrada e a saída ficam em sinal-magnitude, como pede o enunciado, e a conta é feita em complemento a 2.

3.2 Soma e subtração

1. **A subtração vira soma:** $A - B = A + (-B)$. Trocar o sinal de B em sinal-magnitude é só inverter o bit de sinal: $\text{signal_B}' = \text{signal_B} \oplus S0$. Na soma ($S0 = 0$) o sinal fica; na subtração ($S0 = 1$), inverte.
2. **SM $\rightarrow$ C2 (comp2):** se o número é positivo, a magnitude passa direto; se é negativo, sai o C2 da magnitude. O resultado é um número em C2 de 5 bits (-15 a $+15$). O -0 (10000) sai como 00000 ; caso contrário, viraria -16 .
3. **Soma em 6 bits (somador6):** a soma de dois números de -15 a $+15$ vai de -30 a $+30$, que não cabe em 5 bits. Os operandos são estendidos para 6 bits repetindo o bit de sinal e somados por seis somadores completos em cascata. Em 6 bits não há overflow.
4. **C2 $\rightarrow$ SM (c2_para_sm):** se o resultado é positivo, já está em sinal-magnitude. Se é negativo, a magnitude é o C2 dos 5 bits de baixo; o sinal continua no bit 5.

Exemplo, $(-9) - (+12)$:

Tabela 3 – soma e subtração

Passo	A	B	Comentário
Entrada (SM)	11001	01100	-9 e $+12$
Inverte o sinal de B	11001	11100	-9 e -12
comp2 (C2, 5 bits)	10111	10100	-9 e -12 em C2
Estende para 6 bits	110111	110100	repete o bit de sinal
somador6	101011		-21 em C2
c2_para_sm	110101		sinal 1, magnitude $10101 = 21 \rightarrow -21$

3.3 Complemento a 2 de B

A operação 010 usa o próprio comp2 aplicado a B: B positivo sai igual; B negativo sai em C2.

A saída de 5 bits é estendida para 6 bits repetindo o sinal: $F = \{OS, OS, O3, O2, O1, O0\}$.

Exemplo: $B = -3 (10011) \rightarrow C2 = 11101 \rightarrow F = 111101$.

3.4 Comparações em sinal-magnitude

Comparar em sinal-magnitude é comparar os sinais primeiro e as magnitudes depois:

Tabela 4 – comparações em sinal-magnitude

Sinal de A	Sinal de B	A > B quando...
+	+	$ A > B $
-	-	$ A < B $ (o de menor magnitude é o maior)
+	-	sempre, exceto se os dois forem zero ($+0 = -0$)
-	+	nunca

Como $A < B$ é o mesmo que $B > A$, a ULA usa uma segunda instância do comp_maior com A e B trocados, sem precisar de um “comparador menor”.

3.5 Displays de 7 segmentos

Os displays da DE2-115 são de anodo comum: cada segmento **acende com nível lógico 0**. Os segmentos são indexados de 0 a 6 na ordem a, b, c, d, e, f, g (a no topo, g no meio).

Tabela 5 – displays de 7 segmentos

Dígito	a b c d e f g	Dígito	a b c d e f g
0	0000001	5	0100100
1	1001111	6	0100000
2	0010010	7	0001111
3	0000110	8	0000000
4	1001100	9	0000100

Em vez de converter o número para BCD e usar um decodificador BCD $\rightarrow$ 7 segmentos, o projeto usa **decodificadores diretos do binário para cada dígito**: um bloco gera os segmentos da dezena e outro os da unidade, a partir dos 4 bits ($|A|, |B|$) ou 5 bits ($|F|$) da magnitude. Cada segmento é uma soma de produtos tirada da tabela verdade e simplificada por mapa de Karnaugh. A dezena mostra 0 abaixo de 10.

3.6 Decisões de projeto

- **Zero negativo:** $+0 (00000)$ e $-0 (10000)$ são iguais. O comp2 converte -0 em $+0$; nas comparações, comparador_igual dá 1 e comp_maior dá 0 nos dois sentidos.

- **Soma e subtração em C2 de 6 bits:** evita overflow (-30 a $+30$) e usa um único somador para as duas operações.
- **Operação 010:** $F = C2$ de B em 5 bits, estendido para 6 bits repetindo o sinal.
- **Comparações:** $F = 000000$; só o STATUS vale.
- **AND e XOR:** bit a bit sobre os 5 bits, incluindo o sinal ($F5 = SA \text{ op } SB$); $F4 = 0$.
- **Displays de F:** acesos só na soma e na subtração; o sinal de F aparece como “–” (segmento g) no HEX2 e no LED LEDG5.

4 Projeto dos módulos

Para cada módulo são apresentados o funcionamento, a tabela verdade, as equações (com os mapas de Karnaugh, quando a simplificação se aplica), o circuito projetado no Quartus e a simulação (*waveform*). Nos blocos com muitas entradas, em que a tabela completa teria centenas ou milhares de linhas, é mostrada uma forma compacta e casos representativos. Nas equações, ' indica NOT, $\cdot$ AND, $+$ OR, $\oplus$ XOR e $\odot$ XNOR.

4.1 Blocos comuns

4.1.1 mux2x1

Funcionamento

Multiplexador 2:1 de 1 bit. Com $S = 0$ a saída Y copia A; com $S = 1$, copia B. É usado no comp2 e no c2_para_sm para escolher entre um número e o seu complemento a 2.

- **Entradas:** A, B, S
- **Saídas:** Y

Tabela verdade

Tabela 6 – mux2x1: tabela verdade

S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Equações

- $Y = A \cdot S' + B \cdot S$

Mapas de Karnaugh

Tabela 7 – mux2x1: mapa de Karnaugh

S \ AB	00	01	11	10
0	0	0	1	1
1	0	1	1	0

Grupos: $A \cdot S'$ (linha $S = 0$, colunas com $A = 1$) e $B \cdot S$ (linha $S = 1$, colunas com $B = 1$).

Circuito

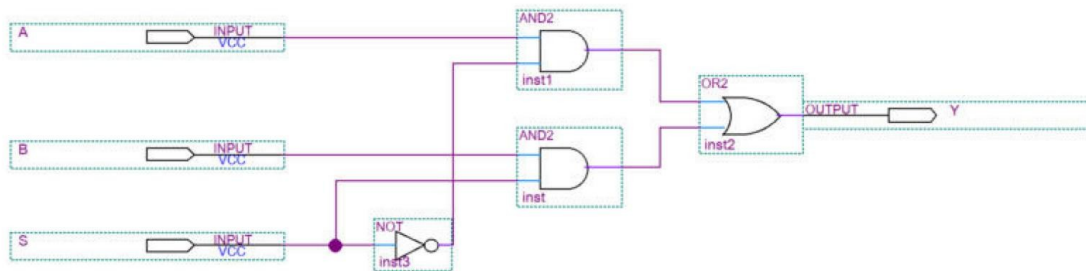


Figura 3 – Circuito do mux2x1 (lib/mux2x1.bdf)

Simulação

[inserir figura: testes/mux2x1/mux2x1_simulacao.png]

Figura 4 – Simulação do mux2x1 (mux2x1.vwf)

4.1.2 mux4x1

Funcionamento

Multiplexador 4:1 de 1 bit: o seletor $S[1..0]$ escolhe qual das quatro entradas vai para y_i . O `mux_saida` usa seis deles, um por bit de F. A tabela completa tem 64 linhas; abaixo, a forma compacta (cada linha vale para qualquer valor das entradas não seleccionadas).

- **Entradas:** $I[3..0]$, $S[1..0]$
- **Saídas:** y_i

Tabela verdade

Tabela 8 – mux4x1: tabela verdade

S[1]	S[0]	y_i
0	0	$I[0]$
0	1	$I[1]$
1	0	$I[2]$
1	1	$I[3]$

Equações

$$y_i = I_0 \cdot S_1' \cdot S_0' + I_1 \cdot S_1' \cdot S_0 + I_2 \cdot S_1 \cdot S_0' + I_3 \cdot S_1 \cdot S_0$$

Cada termo é uma porta AND que só deixa passar a entrada cujo índice é igual a S; a porta OR junta os quatro termos.

Circuito

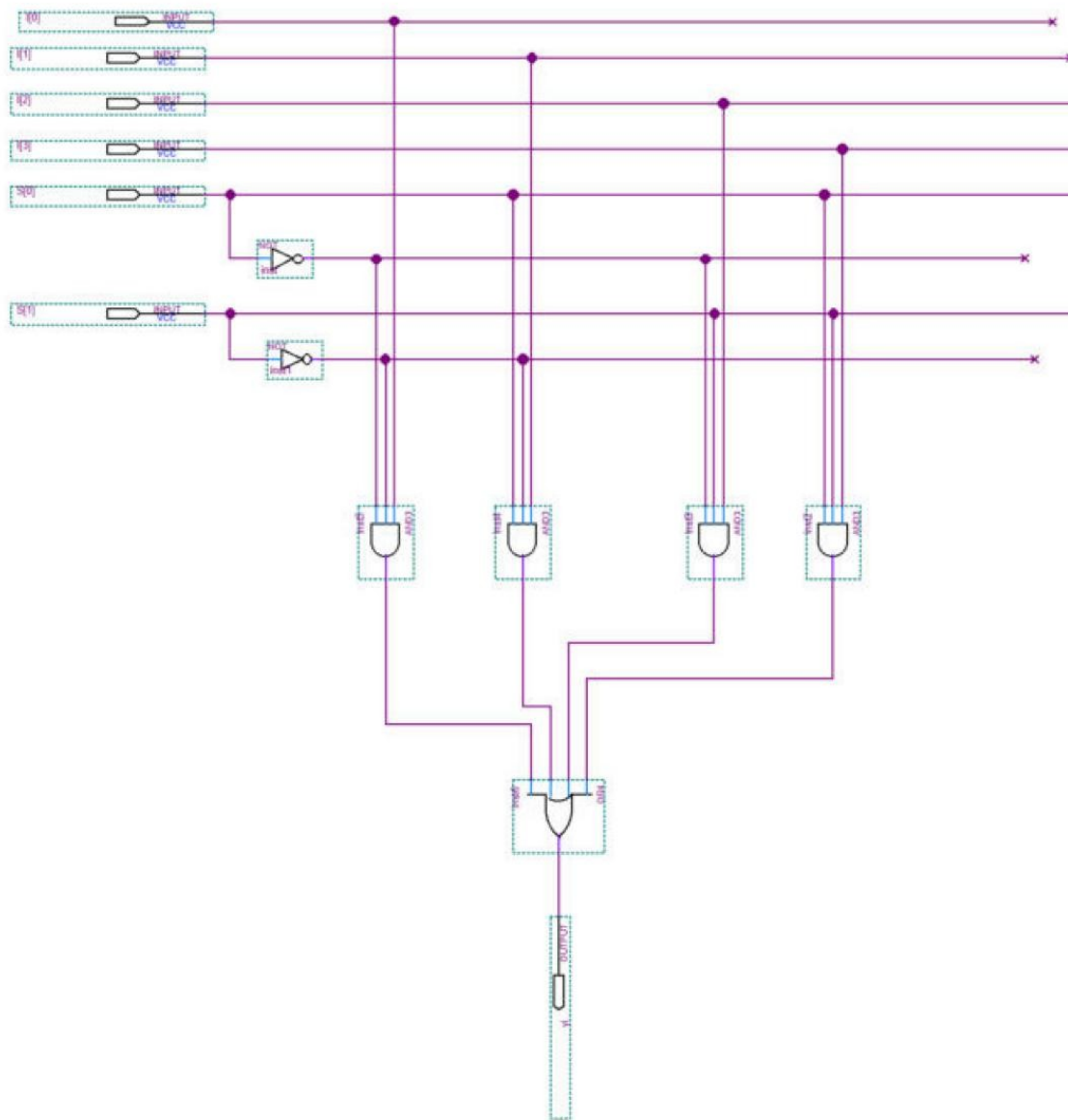


Figura 5 – Circuito do mux4x1 (lib/mux4x1.bdf)

Simulação

[inserir figura: testes/mux4x1/mux4x1_simulacao.png]

Figura 6 – Simulação do mux4x1 (mux4x1.vwf)

4.2 Complemento a 2

4.2.1 inversor

Funcionamento

Calcula o complemento a 2 de um número de 4 bits (inverte e soma 1). Em vez de um somador, usa o atalho: da direita para a esquerda, copia os bits até o primeiro 1 (inclusive) e inverte os outros. Assim, cada bit é invertido quando existe algum 1 abaixo dele. É usado pelo comp2.

- **Entradas:** I3, I2, I1, I0
- **Saídas:** F3, F2, F1, F0

Tabela verdade

Tabela 9 – inversor: tabela verdade

I3	I2	I1	I0	F3	F2	F1	F0	I	F (C2)
0	0	0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	1	1	-1
0	0	1	0	1	1	1	0	2	-2
0	0	1	1	1	1	0	1	3	-3
0	1	0	0	1	1	0	0	4	-4
0	1	0	1	1	0	1	1	5	-5
0	1	1	0	1	0	1	0	6	-6
0	1	1	1	1	0	0	1	7	-7
1	0	0	0	1	0	0	0	8	-8
1	0	0	1	0	1	1	1	9	-9
1	0	1	0	0	1	1	0	10	-10
1	0	1	1	0	1	0	1	11	-11
1	1	0	0	0	1	0	0	12	-12
1	1	0	1	0	0	1	1	13	-13
1	1	1	0	0	0	1	0	14	-14
1	1	1	1	0	0	0	1	15	-15

Equações

- $F0 = I0$
- $F1 = I1 \oplus I0$
- $F2 = I2 \oplus (I1 + I0)$
- $F3 = I3 \oplus (I2 + I1 + I0)$

Pelo mapa-K, F1, F2 e F3 não se reduzem bem em soma de produtos (padrão de tabuleiro, típico de XOR); a forma com XOR sai direto da regra “inverte se há um 1 abaixo”.

Circuito

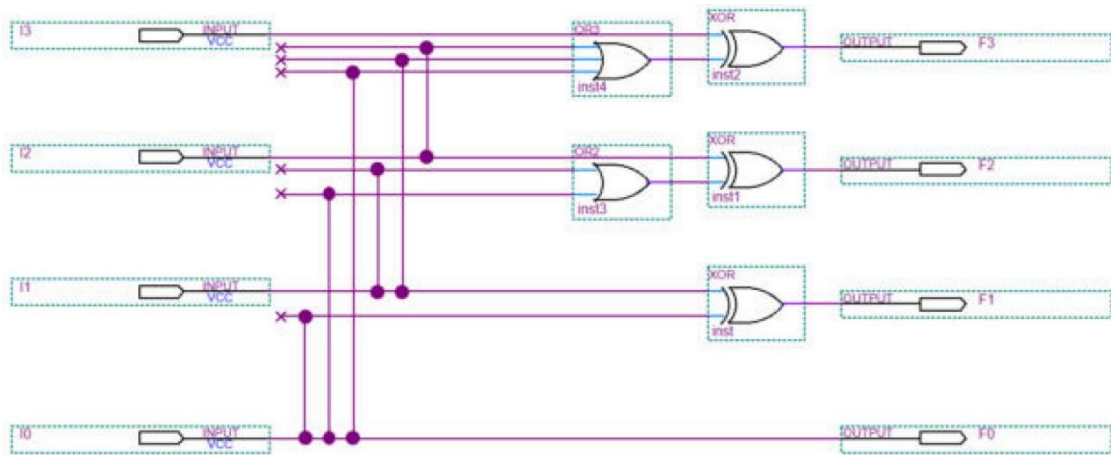


Figura 7 – Circuito do `inversor` (lib/inversor.bdf)

Simulação

[inserir figura: testes/inversor/inversor_simulacao.png]

Figura 8 – Simulação do `inversor` (inversor.vwf)

4.2.2 `inversor5`

Funcionamento

Complemento a 2 de 5 bits, com a mesma regra do `inversor`. É usado no `c2_para_sm`, em que a magnitude do resultado chega a 30 e precisa de 5 bits.

- **Entradas:** `I[4..0]`
- **Saídas:** `O[4..0]`

Tabela verdade

Tabela 10 – `inversor5`: tabela verdade

<code>I[4..0]</code>	<code>I</code>	<code>O[4..0]</code>	<code>I[4..0]</code>	<code>I</code>	<code>O[4..0]</code>
00000	0	00000	10000	16	10000
00001	1	11111	10001	17	01111
00010	2	11110	10010	18	01110
00011	3	11101	10011	19	01101
00100	4	11100	10100	20	01100
00101	5	11011	10101	21	01011
00110	6	11010	10110	22	01010
00111	7	11001	10111	23	01001
01000	8	11000	11000	24	01000
01001	9	10111	11001	25	00111
01010	10	10110	11010	26	00110
01011	11	10101	11011	27	00101
01100	12	10100	11100	28	00100
01101	13	10011	11101	29	00011

I[4..0]	I	O[4..0]	I[4..0]	I	O[4..0]
01110	14	10010	11110	30	00010
01111	15	10001	11111	31	00001

Equações

- $O[0] = I[0]$
- $O[1] = I[1] \oplus I[0]$
- $O[2] = I[2] \oplus (I[1] + I[0])$
- $O[3] = I[3] \oplus (I[2] + I[1] + I[0])$
- $O[4] = I[4] \oplus (I[3] + I[2] + I[1] + I[0])$

Circuito

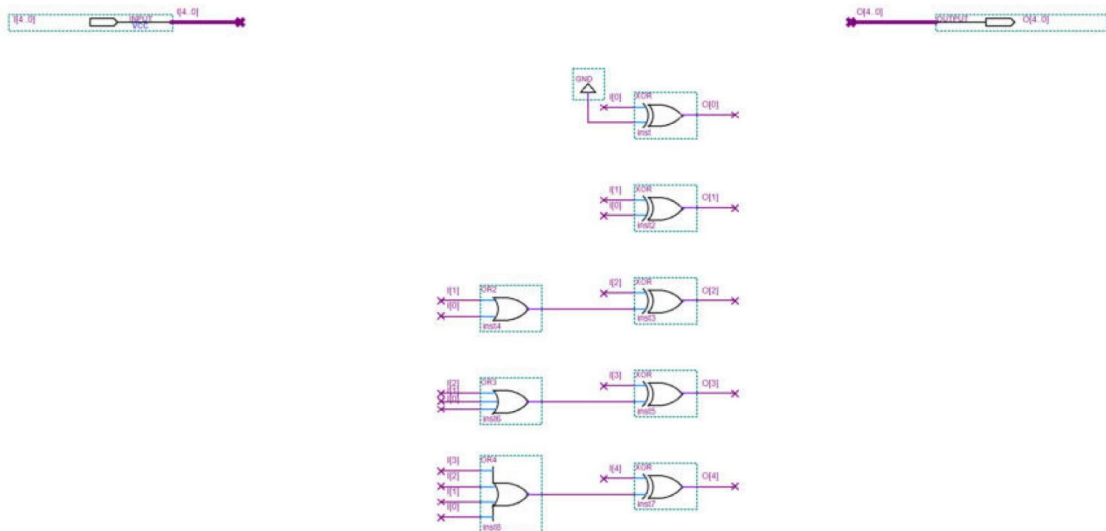


Figura 9 – Circuito do `inversor5` (lib/inversor5.bdf)

Simulação

[inserir figura: testes/inversor5/inversor5_simulacao.png]

Figura 10 – Simulação do `inversor5` (inversor5.vwf)

4.2.3 comp2

Funcionamento

Converte um número de sinal-magnitude para complemento a 2 de 5 bits. Se o número é positivo, a magnitude passa direto; se é negativo, sai o complemento a 2 da magnitude (bloco `inversor`). Quatro `mux2x1` com seletor `IS` fazem essa escolha. O sinal de saída só é 1 quando a magnitude não é zero, então o -0 ($IS = 1, I = 0000$) sai como +0. Na ULA, é usado no somador_subtrator ($\times 2$) e na operação 010.

- **Usa:** `inversor`, `mux2x1` $\times 4$
- **Entradas:** `IS` (sinal), `I[3..0]` (magnitude)
- **Saídas:** `OS`, `O[3..0]` (complemento a 2 de 5 bits)

Tabela verdade

Tabela 11 – comp2: tabela verdade

IS	I[3..0]	Entrada (SM)	OS	O[3..0]	Saída (C2)
0	0000	0	0	0000	0
0	0001	+1	0	0001	+1
0	0010	+2	0	0010	+2
0	0011	+3	0	0011	+3
0	0100	+4	0	0100	+4
0	0101	+5	0	0101	+5
0	0110	+6	0	0110	+6
0	0111	+7	0	0111	+7
0	1000	+8	0	1000	+8
0	1001	+9	0	1001	+9
0	1010	+10	0	1010	+10
0	1011	+11	0	1011	+11
0	1100	+12	0	1100	+12
0	1101	+13	0	1101	+13
0	1110	+14	0	1110	+14
0	1111	+15	0	1111	+15
1	0000	-0	0	0000	0
1	0001	-1	1	1111	-1
1	0010	-2	1	1110	-2
1	0011	-3	1	1101	-3
1	0100	-4	1	1100	-4
1	0101	-5	1	1011	-5
1	0110	-6	1	1010	-6
1	0111	-7	1	1001	-7
1	1000	-8	1	1000	-8
1	1001	-9	1	0111	-9
1	1010	-10	1	0110	-10
1	1011	-11	1	0101	-11
1	1100	-12	1	0100	-12
1	1101	-13	1	0011	-13
1	1110	-14	1	0010	-14
1	1111	-15	1	0001	-15

Equações

- $O[k] = I[k] \cdot IS' + \text{inversor}(I)[k] \cdot IS$ (um mux2x1 por bit)
- $OS = IS \cdot (I3 + I2 + I1 + I0)$

Circuito

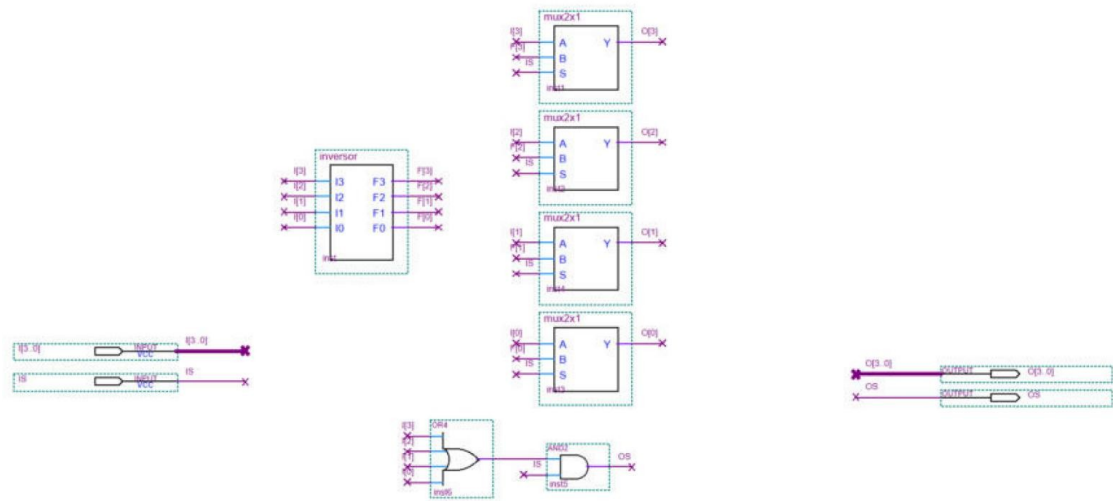


Figura 11 – Circuito do comp2 (lib/comp2.bdf)

Simulação

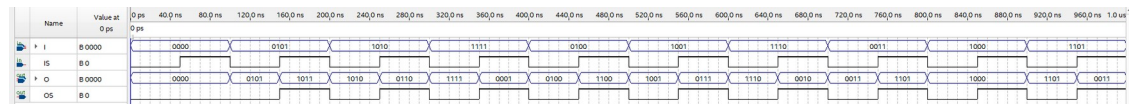


Figura 12 – Simulação do comp2 (comp2.vwf)

4.3 Soma e subtração

4.3.1 somador_completo

Funcionamento

Soma três bits (dois operandos e o vai-um da etapa anterior), gerando o bit de soma S e o vai-um Cout. É a célula do somador6.

- Entradas: A, B, Cin
- Saídas: S, Cout

Tabela verdade

Tabela 12 – somador_completo: tabela verdade

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Equações

- $S = A \oplus B \oplus Cin$
- $Cout = A \cdot B + (A \oplus B) \cdot Cin$

Mapas de Karnaugh

Tabela 13 – somador_completo: mapa de Karnaugh de S

A \ BCin	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Nenhum par de 1s é adjacente (tabuleiro): S não se reduz em soma de produtos e vira $A \oplus B \oplus Cin$.

Tabela 14 – somador_completo: mapa de Karnaugh de Cout

A \ BCin	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Três grupos de dois: $Cout = A \cdot B + A \cdot Cin + B \cdot Cin$. O circuito usa a forma equivalente $A \cdot B + (A \oplus B) \cdot Cin$, que reaproveita a porta XOR de S.

Circuito

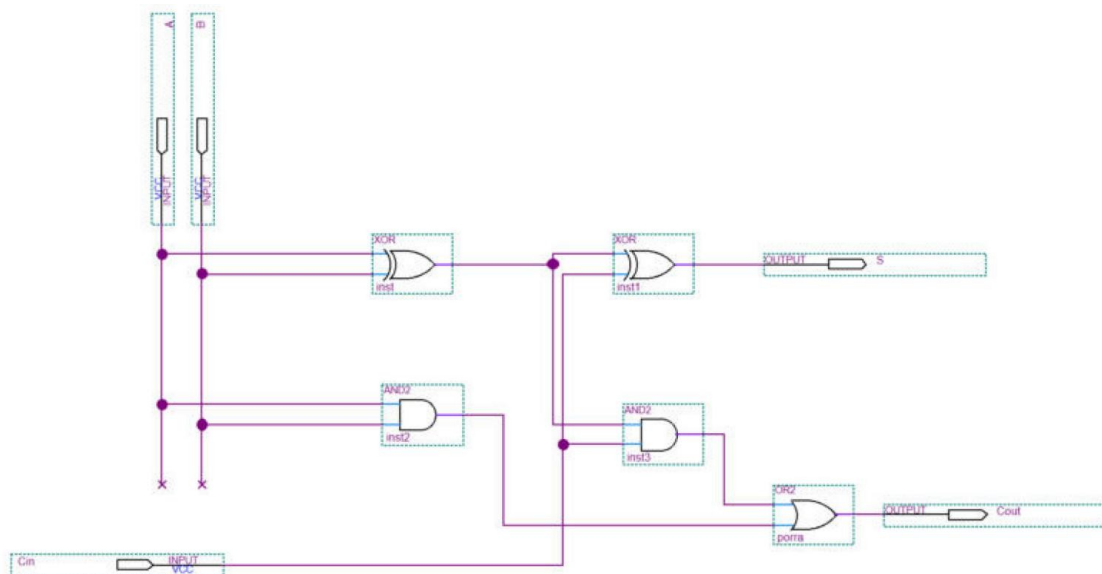


Figura 13 – Circuito do somador_completo (lib/somador_completo.bdf)

Simulação

[inserir figura: testes/somador_completo/somador_completo_simulacao.png]

Figura 14 – Simulação do somador_completo (somador_completo.vwf)

4.3.2 somador6

Funcionamento

Soma dois números em complemento a 2 de 5 bits (−15 a +15, vindos do comp2). Como a soma vai de −30 a +30 e não cabe em 5 bits, os operandos são estendidos para 6 bits repetindo o bit de sinal: o sexto somador_completo recebe A[4] e B[4] de novo. São seis somadores em cascata (*ripple carry*), com vai-um inicial 0; em 6 bits não há overflow. A tabela completa tem 1024 linhas; abaixo, casos representativos (sinais iguais e diferentes, zero e os extremos ±30).

- **Usa:** somador_completo ×6
- **Entradas:** A[4..0], B[4..0] (complemento a 2)
- **Saídas:** O[5..0] = A + B (complemento a 2 de 6 bits)

Tabela verdade

Tabela 15 – somador6: tabela verdade

A[4..0]	A	B[4..0]	B	O[5..0]	O
00000	0	00000	0	000000	0
00101	+5	11101	−3	000010	+2
11011	−5	00011	+3	111110	−2
00111	+7	11001	−7	000000	0
01111	+15	01111	+15	011110	+30
10001	−15	10001	−15	100010	−30
11000	−8	11000	−8	110000	−16
10111	−9	10100	−12	101011	−21
01100	+12	10111	−9	000011	+3
00001	+1	11111	−1	000000	0
11111	−1	11111	−1	111110	−2
01111	+15	10001	−15	000000	0

Equações

Para cada etapa $i = 0 \dots 5$, com $A_5 = A_4$, $B_5 = B_4$ (extensão de sinal) e $C_0 = 0$:

- $O[i] = A[i] \oplus B[i] \oplus C[i]$
- $C[i+1] = A[i] \cdot B[i] + (A[i] \oplus B[i]) \cdot C[i]$

O vai-um final C_6 é descartado.

Circuito

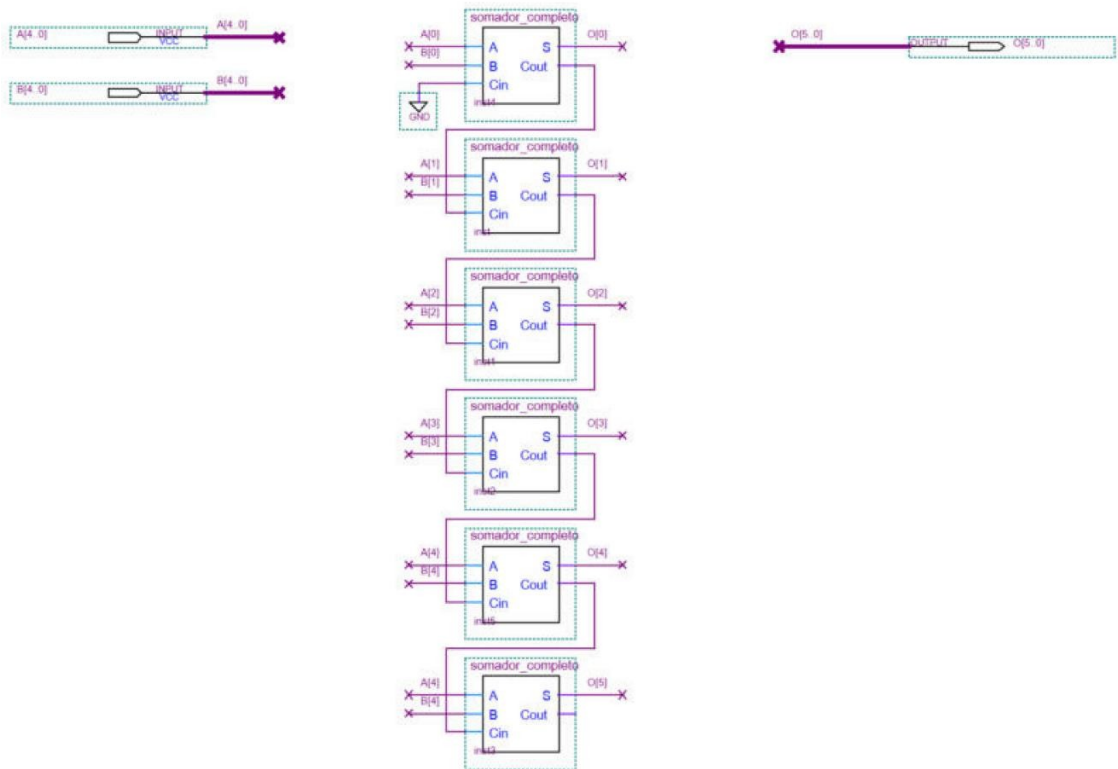


Figura 15 – Circuito do somador6 (lib/somador6.bdf)

Simulação

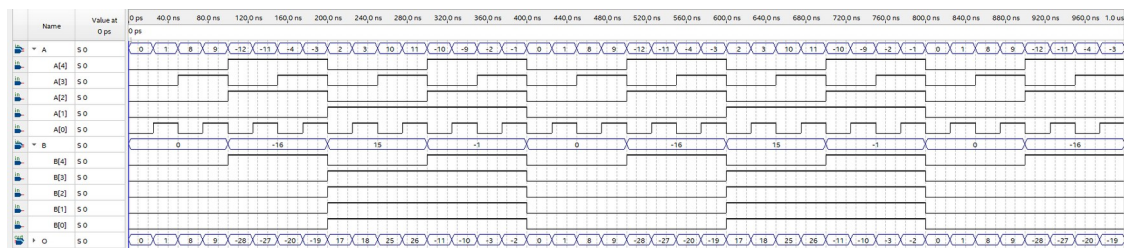


Figura 16 – Simulação do somador6 (somador6.vwf)

4.3.3 c2_para_sm

Funcionamento

Converte o resultado do somador6 de volta para sinal-magnitude, que é o formato pedido para F. O sinal passa direto ($O[5] = I[5]$). Se o número é positivo, a magnitude são os 5 bits de baixo; se é negativo, é o complemento a 2 desses 5 bits (inversor5). Cinco mux2x1 com seletor $I[5]$ fazem a escolha. Na ULA só aparecem valores de -30 a +30; a entrada -32 (100000) não ocorre (X).

- **Usa:** inversor5, mux2x1 x5
- **Entradas:** $I[5..0]$ (complemento a 2, saída do somador6)
- **Saídas:** $O[5..0]$ (sinal-magnitude: $O[5] = \text{sinal}$)

Tabela verdade

Tabela 16 – c2_para_sm: tabela verdade

I[5..0]	Valor	O[5..0]	I[5..0]	Valor	O[5..0]
000000	0	000000	100000	-32	X
000001	+1	000001	100001	-31	111111
000010	+2	000010	100010	-30	111110
000011	+3	000011	100011	-29	111101
000100	+4	000100	100100	-28	111100
000101	+5	000101	100101	-27	111011
000110	+6	000110	100110	-26	111010
000111	+7	000111	100111	-25	111001
001000	+8	001000	101000	-24	111000
001001	+9	001001	101001	-23	110111
001010	+10	001010	101010	-22	110110
001011	+11	001011	101011	-21	110101
001100	+12	001100	101100	-20	110100
001101	+13	001101	101101	-19	110011
001110	+14	001110	101110	-18	110010
001111	+15	001111	101111	-17	110001
010000	+16	010000	110000	-16	110000
010001	+17	010001	110001	-15	101111
010010	+18	010010	110010	-14	101110
010011	+19	010011	110011	-13	101101
010100	+20	010100	110100	-12	101100
010101	+21	010101	110101	-11	101011
010110	+22	010110	110110	-10	101010
010111	+23	010111	110111	-9	101001
011000	+24	011000	111000	-8	101000
011001	+25	011001	111001	-7	100111
011010	+26	011010	111010	-6	100110
011011	+27	011011	111011	-5	100101
011100	+28	011100	111100	-4	100100
011101	+29	011101	111101	-3	100011
011110	+30	011110	111110	-2	100010
011111	+31	011111	111111	-1	100001

Equações

- $O[5] = I[5]$
- $O[k] = I[k] \cdot I5' + \text{inversor5}(I[4..0])[k] \cdot I5$, para $k = 0 \dots 4$

Circuito

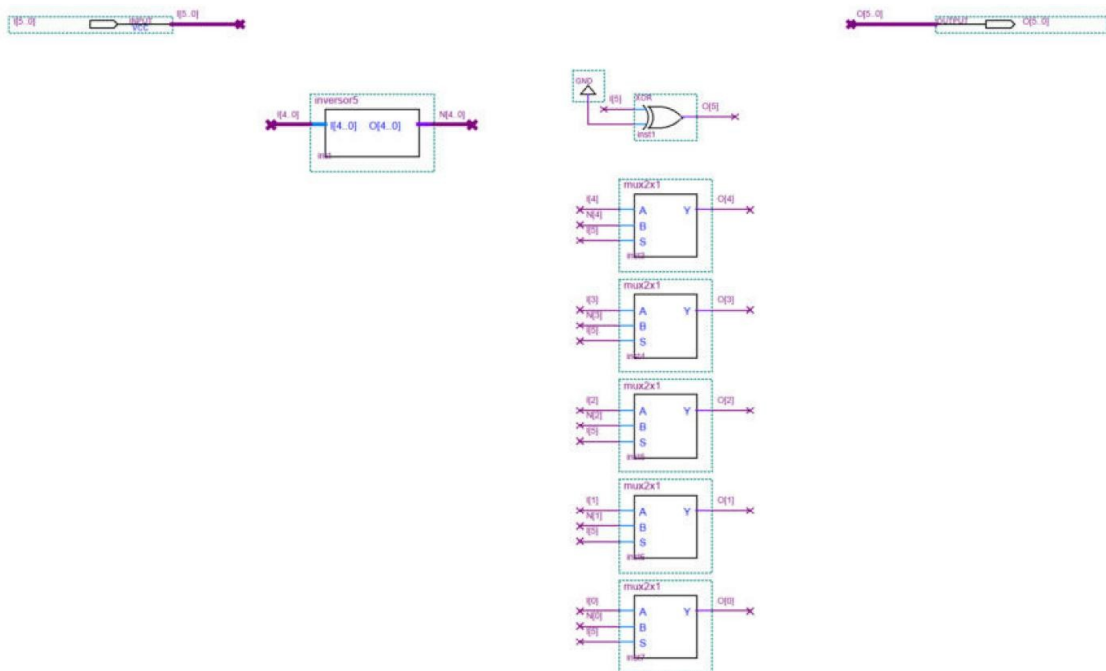


Figura 17 – Circuito do c2_para_sm (lib/c2_para_sm.bdf)

Simulação

[inserir figura: testes/c2_para_sm/c2_para_sm_simulacao.png]

Figura 18 – Simulação do c2_para_sm (c2_para_sm.vwf)

4.3.4 somador_subtrator

Funcionamento

Faz a soma e a subtração da ULA ($S = 000$ e 001 , com $\text{signal_op} = S[0]$). A subtração vira soma: $A - B = A + (-B)$, e trocar o sinal de B em sinal-magnitude é só inverter o bit de sinal ($\text{signal_b} \oplus \text{signal_op}$). Depois, os dois operandos passam por comp2 (SM $\rightarrow$ C2), são somados no somador6 e o resultado volta para sinal-magnitude no c2_para_sm. A tabela completa tem 512 linhas; abaixo, casos representativos.

- **Usa:** comp2 $\times 2$, XOR, somador6, c2_para_sm
- **Entradas:** signal_a, w[3..0] (A), signal_b, x[3..0] (B), signal_op (0 = soma, 1 = subtração)
- **Saídas:** R[5..0] = $A \pm B$ em sinal-magnitude

Tabela verdade

Tabela 17 – somador_subtrator: tabela verdade

signal_op	A (SM)	B (SM)	Conta	R[5..0]	R
0	00101	10011	(+5) + (-3)	000010	+2
0	10101	00011	(-5) + (+3)	100010	-2

sinal_op	A (SM)	B (SM)	Conta	R[5..0]	R
1	00010	00101	(+2) - (+5)	100011	-3
1	11001	01100	(-9) - (+12)	110101	-21
0	11000	11000	(-8) + (-8)	110000	-16
0	11111	11111	(-15) + (-15)	111110	-30
0	01111	01111	(+15) + (+15)	011110	+30
1	01111	11111	(+15) - (-15)	011110	+30
1	00111	00111	(+7) - (+7)	000000	0
0	00110	10110	(+6) + (-6)	000000	0
1	10100	10100	(-4) - (-4)	000000	0
0	00000	00000	(0) + (0)	000000	0
0	10000	00000	(-0) + (+0)	000000	0

Equações

- $\text{sinal_b}' = \text{sinal_b} \oplus \text{sinal_op}$
- $R = \text{c2_para_sm}(\text{somador6}(\text{comp2}(\text{sinal_a}, W), \text{comp2}(\text{sinal_b}', X)))$

Circuito

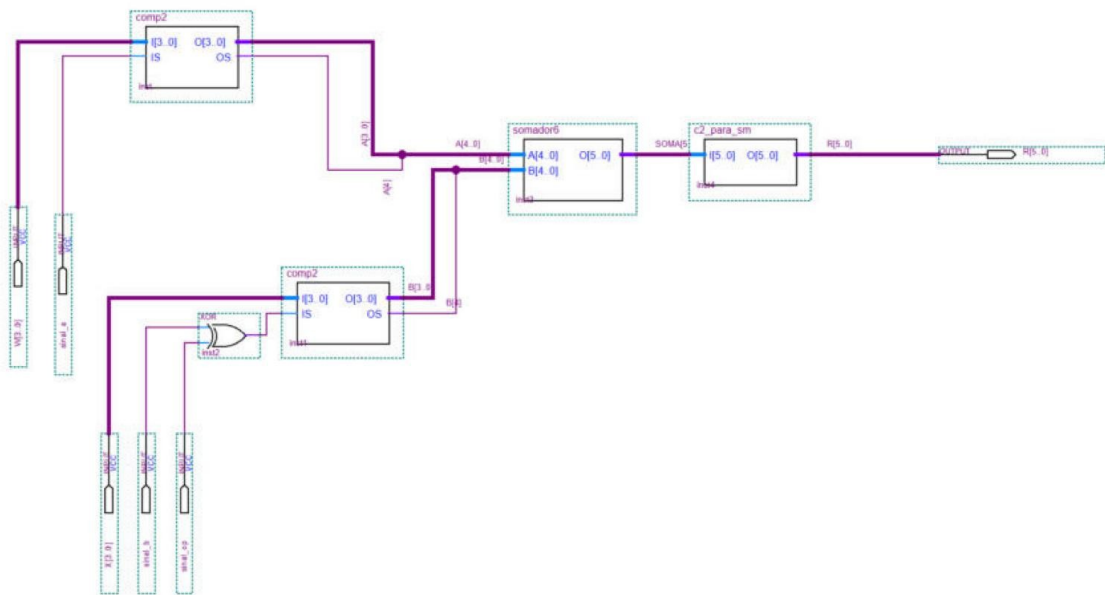


Figura 19 – Circuito do somador_subtrator (lib/somador_subtrator.bdf)

Simulação

[inserir figura: testes/somador_subtrator/somador_subtrator_simulacao.png]

Figura 20 – Simulação do somador_subtrator (somador_subtrator.vwf)

4.4 Operações lógicas

4.4.1 op_and

Funcionamento

Operação 110: A AND B. A operação é bit a bit sobre os 5 bits de entrada: as magnitudes vão para $F[3..0]$ e os sinais, para $F[5]$. $F[4]$ fica sempre em 0, porque a magnitude de entrada tem só 4 bits. A tabela completa tem 1024 linhas; como cada bit é independente, basta a tabela de um bit e alguns exemplos.

- **Entradas:** SA, $A[3..0]$, SB, $B[3..0]$
- **Saídas:** $F[5..0]$

Tabela verdade

Tabela 18 – op_and: tabela verdade (um bit)

a	b	$a \cdot b$
0	0	0
0	1	0
1	0	0
1	1	1

Tabela 19 – op_and: tabela verdade (exemplos)

A (SM)	B (SM)	F[5..0]
00101	00011	000001
10101	10011	100001
10101	00011	000001
01111	01001	001001
11100	11010	101000
00000	10111	000000

Equações

- $F[k] = A[k] \cdot B[k]$, para $k = 0 \dots 3$
- $F[4] = 0$
- $F[5] = SA \cdot SB$

Circuito

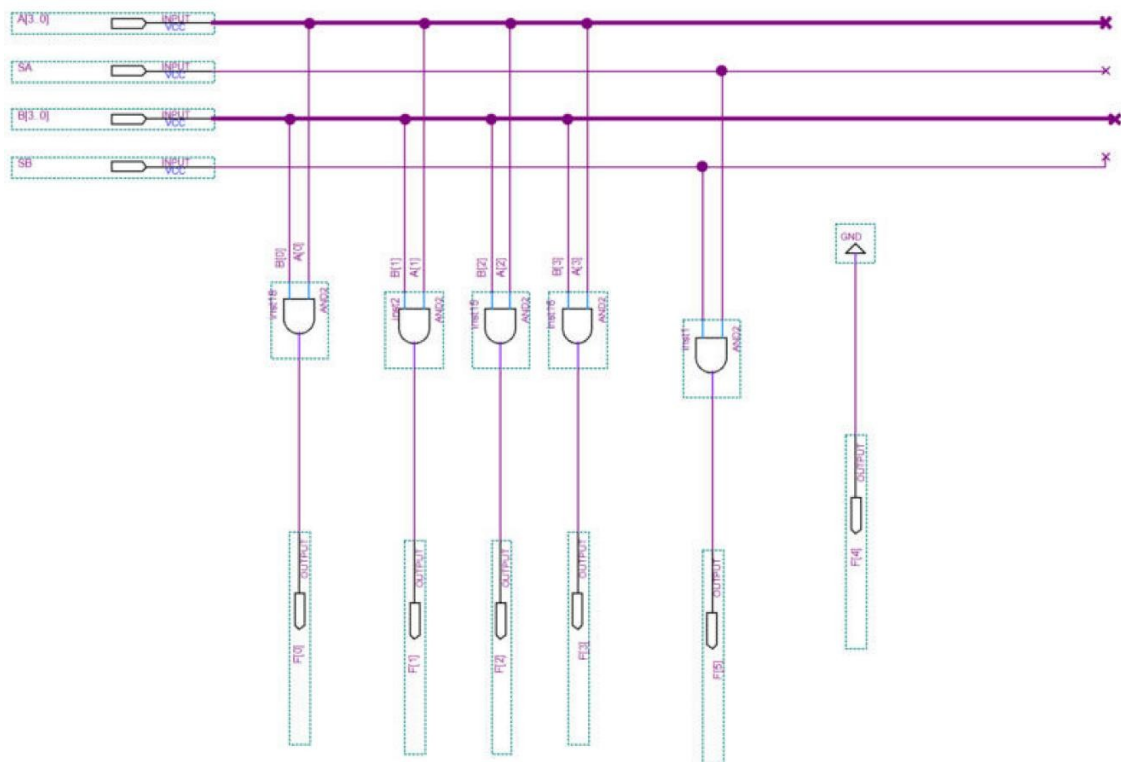


Figura 21 – Circuito do op_and (lib/op_and.bdf)

Simulação

[inserir figura: testes/op_and/op_and_simulacao.png]

Figura 22 – Simulação do op_and (op_and.vwf)

4.4.2 op_xor

Funcionamento

Operação 111: A XOR B. A operação é bit a bit sobre os 5 bits de entrada: as magnitudes vão para F[3..0] e os sinais, para F[5]. F[4] fica sempre em 0, porque a magnitude de entrada tem só 4 bits. A tabela completa tem 1024 linhas; como cada bit é independente, basta a tabela de um bit e alguns exemplos.

- **Entradas:** SA, A[3..0], SB, B[3..0]
- **Saídas:** F[5..0]

Tabela verdade

Tabela 20 – op_xor: tabela verdade (um bit)

a	b	$a \oplus b$
0	0	0
0	1	1

a	b	$a \oplus b$
1	0	1
1	1	0

Tabela 21 – op_xor: tabela verdade (exemplos)

A (SM)	B (SM)	F[5..0]
00101	00011	000110
10101	10011	000110
10101	00011	100110
01111	01001	000110
11100	11010	000110
00000	10111	100111

Equações

- $F[k] = A[k] \oplus B[k]$, para $k = 0 \dots 3$
- $F[4] = 0$
- $F[5] = SA \oplus SB$

Circuito

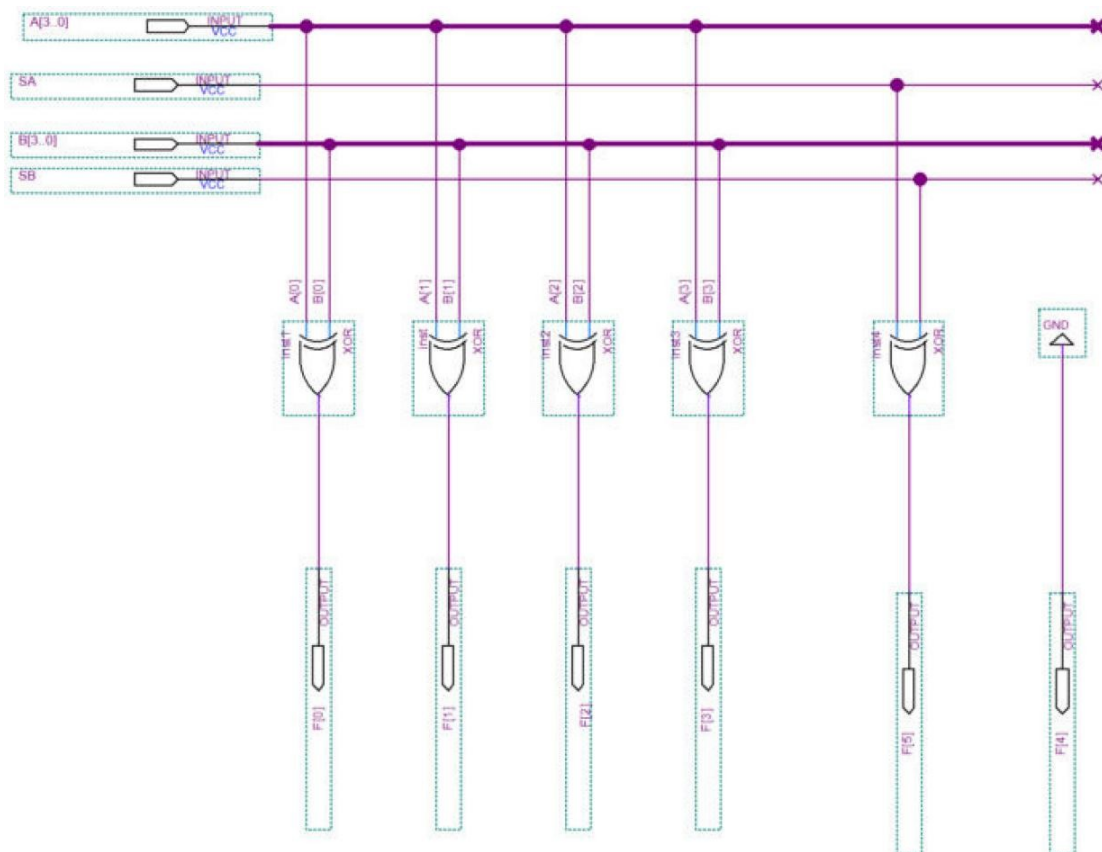


Figura 23 – Circuito do op_xor (lib/op_xor.bdf)

Simulação

[inserir figura: testes/op_xor/op_xor_simulacao.png]

Figura 24 – Simulação do op_xor (op_xor.vwf)

4.5 Comparações

4.5.1 comp_mag

Funcionamento

Compara duas magnitudes de 4 bits, do bit mais alto para o mais baixo: o primeiro bit em que A e B diferem decide. Se todos são iguais, A não é maior ($O = 0$). É a base do comp_maior. A tabela completa tem 256 linhas; abaixo, a forma compacta (X = qualquer valor) e alguns exemplos.

- **Entradas:** A[3..0], B[3..0] (magnitudes, sem sinal)
- **Saídas:** O = 1 quando $|A| > |B|$

Tabela verdade

Tabela 22 – comp_mag: tabela verdade

A3 B3	A2 B2	A1 B1	A0 B0	O
1 0	X	X	X	1
0 1	X	X	X	0
iguais	1 0	X	X	1
iguais	0 1	X	X	0
iguais	iguais	1 0	X	1
iguais	iguais	0 1	X	0
iguais	iguais	iguais	1 0	1
iguais	iguais	iguais	0 1	0
iguais	iguais	iguais	iguais	0

Tabela 23 – comp_mag: tabela verdade (exemplos)

A[3..0]	B[3..0]	Comparação	O
1001	0111	$9 > 7$	1
0111	1001	$7 > 9$	0
1100	1100	$12 > 12$	0
0101	0100	$5 > 4$	1
0000	0000	$0 > 0$	0
1111	0000	$15 > 0$	1
0000	0001	$0 > 1$	0

Equações

Com $e_k = A_k \odot B_k$ (XNOR: bits iguais):

- $O = A_3 \cdot B_3' + e_3 \cdot A_2 \cdot B_2' + e_3 \cdot e_2 \cdot A_1 \cdot B_1' + e_3 \cdot e_2 \cdot e_1 \cdot A_0 \cdot B_0'$

Circuito

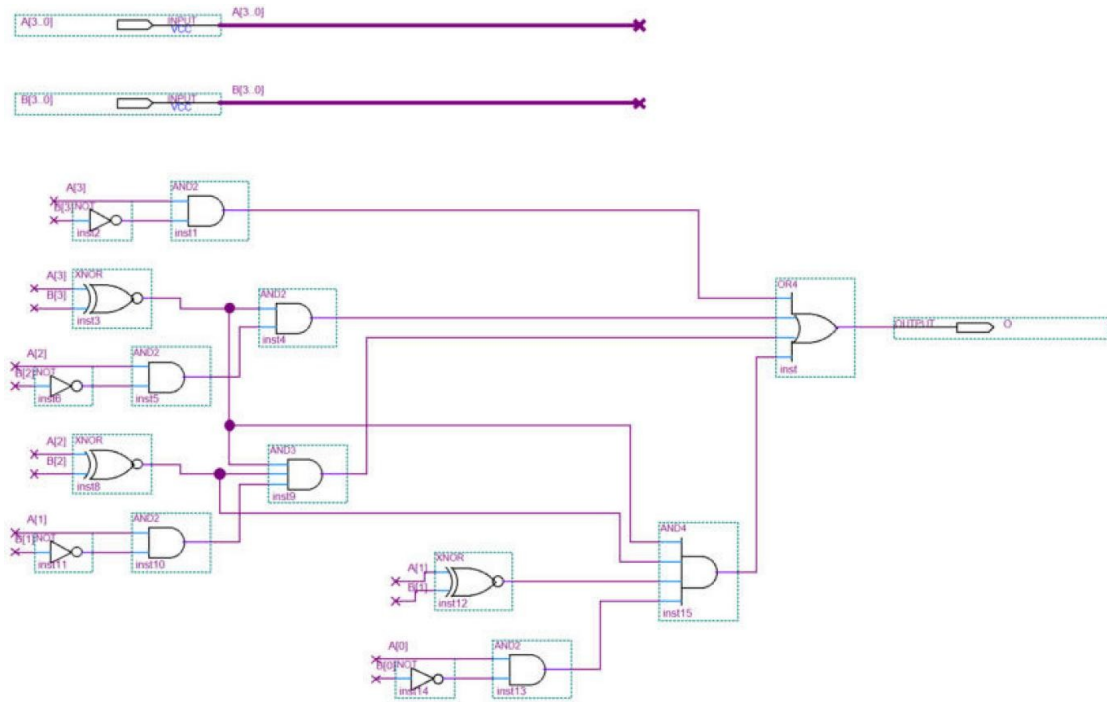


Figura 25 – Circuito do comp_mag (lib/comp_mag.bdf)

Simulação

[inserir figura: testes/comp_mag/comp_mag_simulacao.png]

Figura 26 – Simulação do comp_mag (comp_mag.vwf)

4.5.2 comp_maior

Funcionamento

Diz se $A > B$ em sinal-magnitude. Compara os sinais primeiro e as magnitudes depois: entre dois positivos, ganha a maior magnitude; entre dois negativos, a menor; positivo é maior que negativo, exceto quando os dois são zero ($+0 = -0$). Usa dois comp_mag: $GT = \text{comp_mag}(A, B)$ e $LT = \text{comp_mag}(B, A)$. Na ULA há duas instâncias: comp_maior(A, B) para $A > B$ e comp_maior(B, A) para $A < B$. A tabela completa tem 1024 linhas; abaixo, a forma compacta por sinais e exemplos.

- **Usa:** comp_mag $\times 2$
- **Entradas:** SA, A[3..0], SB, B[3..0] (sinal-magnitude)
- **Saídas:** 0 = 1 quando $A > B$

Tabela verdade

Tabela 24 – comp_maior: tabela verdade

SA	SB	O
0	0	
1	1	
0	1	1, exceto se A = B = 0 (+0 = -0)
1	0	0 (nunca)

Tabela 25 – comp_maior: tabela verdade (exemplos)

SA	A[3..0]	SB	B[3..0]	Comparação	O
0	0101	0	0011	+5 > +3	1
0	0011	0	0101	+3 > +5	0
0	0010	1	0101	+2 > -5	1
1	0010	1	0101	-2 > -5	1
1	0101	1	0010	-5 > -2	0
1	0111	0	0001	-7 > +1	0
0	0000	0	0000	+0 > +0	0
0	0000	1	0000	+0 > -0	0
1	0000	0	0000	-0 > +0	0
0	0100	0	0100	+4 > +4	0

Equações

Com $GT = \text{comp_mag}(A, B)$, $LT = \text{comp_mag}(B, A)$ e $NZB = B3 + B2 + B1 + B0$ ($B \neq 0$):

$$O = SA' \cdot (GT + SB \cdot NZB) + SA \cdot SB \cdot LT$$

Circuito

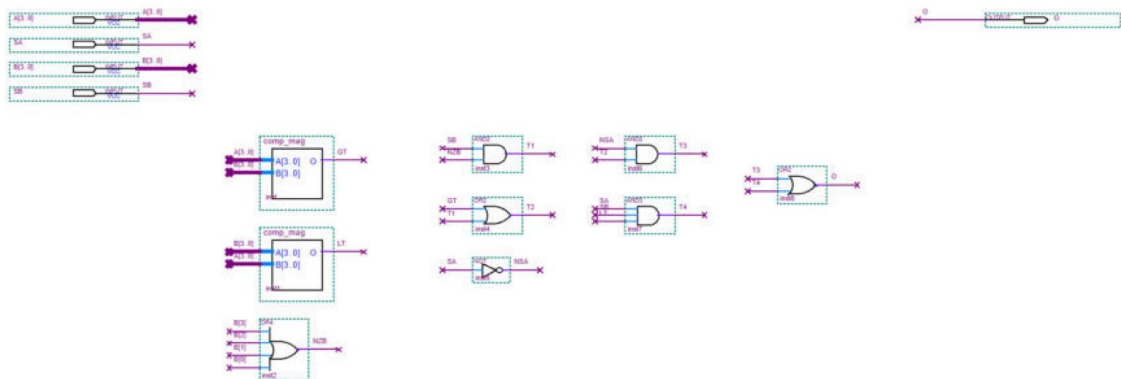


Figura 27 – Circuito do comp_maior (lib/comp_maior.bdf)

Simulação

[inserir figura: testes/comp_maior/comp_maior_simulacao.png]

Figura 28 – Simulação do comp_maior (comp_maior.vwf)

4.5.3 comparador_igual

Funcionamento

Operação 011: diz se $A = B$. As magnitudes têm de ser iguais e, se não forem zero, os sinais também; assim $+0$ e -0 são considerados iguais. A tabela completa tem 1024 linhas; abaixo, a forma compacta e exemplos.

- **Entradas:** $A[4..0]$, $B[4..0]$ (sinal-magnitude, bit 4 = sinal)
- **Saídas:** $F = 1$ quando $A = B$

Tabela verdade

Tabela 26 – comparador_igual: tabela verdade

Magnitudes	Sinais	F
diferentes	X	0
iguais e = 0000	X	1
iguais e $\neq$ 0000	iguais	1
iguais e $\neq$ 0000	diferentes	0

Tabela 27 – comparador_igual: tabela verdade (exemplos)

A[4..0]	B[4..0]	Comparação	F
00101	00101	$+5 = +5$	1
00101	10101	$+5 = -5$	0
10000	00000	$-0 = +0$	1
10000	10000	$-0 = -0$	1
01100	01101	$+12 = +13$	0
11111	11111	$-15 = -15$	1

Equações

- $F = [(A3 \oplus B3) + (A2 \oplus B2) + (A1 \oplus B1) + (A0 \oplus B0) + (B3 + B2 + B1 + B0) \cdot (A4 \oplus B4)]'$

Circuito

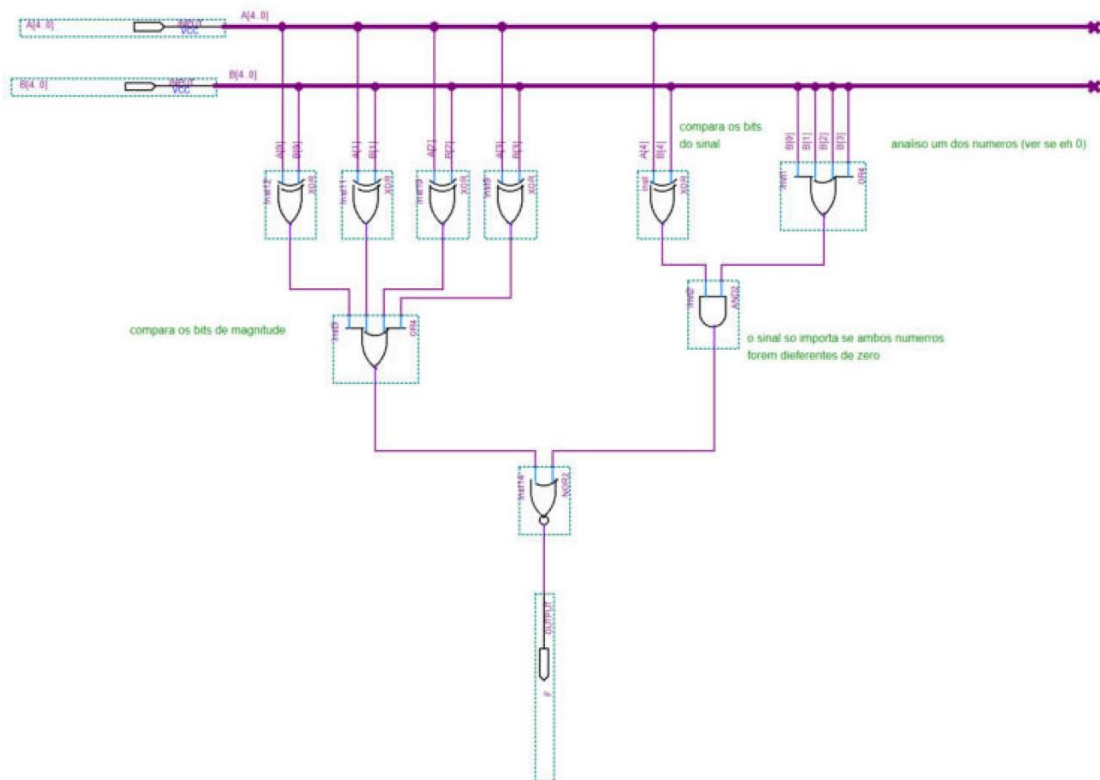


Figura 29 – Circuito do comparador_igual (lib/comparador_igual.bdf)

Simulação

[inserir figura: testes/comparador_igual/comparador_igual_simulacao.png]

Figura 30 – Simulação do comparador_igual (comparador_igual.vwf)

4.5.4 decodificador_comparadores

Funcionamento

Traduz o código da operação no seletor do mux_comparadores: F2 F1 = 10 em 011 (A = B), 01 em 100 (A > B), 11 em 101 (A < B) e 00 nas operações que não são comparações (STATUS = 0).

- **Entradas:** S3, S2, S1 (= S[2], S[1], S[0])
- **Saídas:** F2, F1 (seletor do mux_comparadores)

Tabela verdade

Tabela 28 – decodificador_comparadores: tabela verdade

S3	S2	S1	Operação	F2	F1
0	0	0	A + B	0	0
0	0	1	A - B	0	0
0	1	0	C2 de B	0	0
0	1	1	A = B	1	0

S3	S2	S1	Operação	F2	F1
1	0	0	A > B	0	1
1	0	1	A < B	1	1
1	1	0	A AND B	0	0
1	1	1	A XOR B	0	0

Equações

- $F1 = S3 \cdot S2'$
- $F2 = S3' \cdot S2 \cdot S1 + S3 \cdot S2' \cdot S1 = S1 \cdot (S2 \oplus S3)$

Mapas de Karnaugh

Tabela 29 – decodificador_comparadores: mapa de Karnaugh de F1

S3 \ S2S1	00	01	11	10
0	0	0	0	0
1	1	1	0	0

Um grupo de dois: $S3 \cdot S2'$.

Tabela 30 – decodificador_comparadores: mapa de Karnaugh de F2

S3 \ S2S1	00	01	11	10
0	0	0	1	0
1	0	1	0	0

Dois 1s isolados: $S3' \cdot S2 \cdot S1 + S3 \cdot S2' \cdot S1$, que se fatora em $S1 \cdot (S2 \oplus S3)$.

Circuito

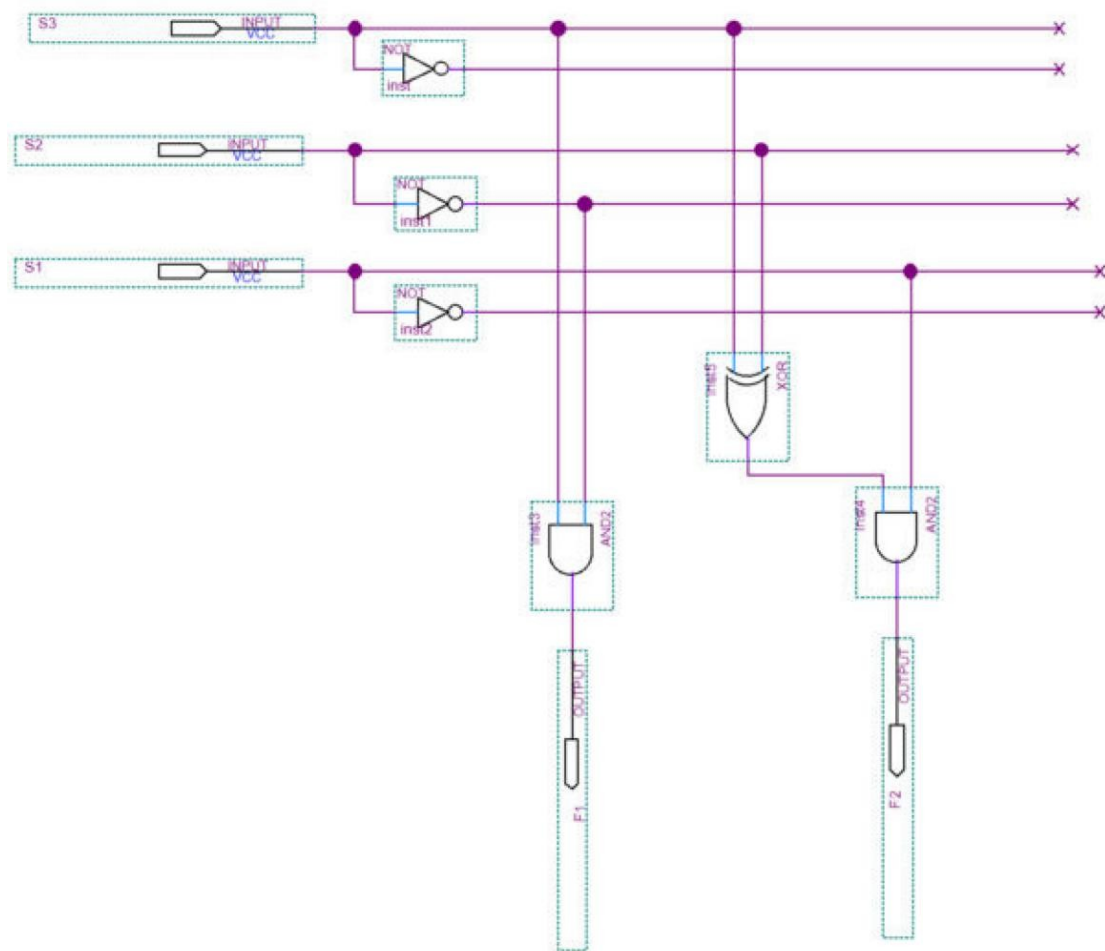


Figura 31 – Circuito do decodificador_comparadores (lib/decodificador_comparadores.bdf)

Simulação

[inserir figura:
testes/decodificador_comparadores/decodificador_comparadores_simulacao.png]

Figura 32 – Simulação do decodificador_comparadores (decodificador_comparadores.vwf)

4.5.5 mux_comparadores

Funcionamento

Escolhe qual comparação vai para o LED de STATUS, conforme o seletor gerado pelo decodificador_comparadores. Com $F2 \ F1 = 00$, $STATUS = 0$. A tabela completa tem 32 linhas; abaixo, a forma compacta.

- **Entradas:** compIgual, compMaior, compMenor, F2, F1
- **Saídas:** STATUS

Tabela verdade

Tabela 31 – mux_comparadores: tabela verdade

F2	F1	STATUS	Operação (S)
0	0	0	outras
0	1	compMaior	100
1	0	compIgual	011
1	1	compMenor	101

Equações

- $STATUS = compIgual \cdot F2 \cdot F1' + compMaior \cdot F2' \cdot F1 + compMenor \cdot F2 \cdot F1$

Circuito

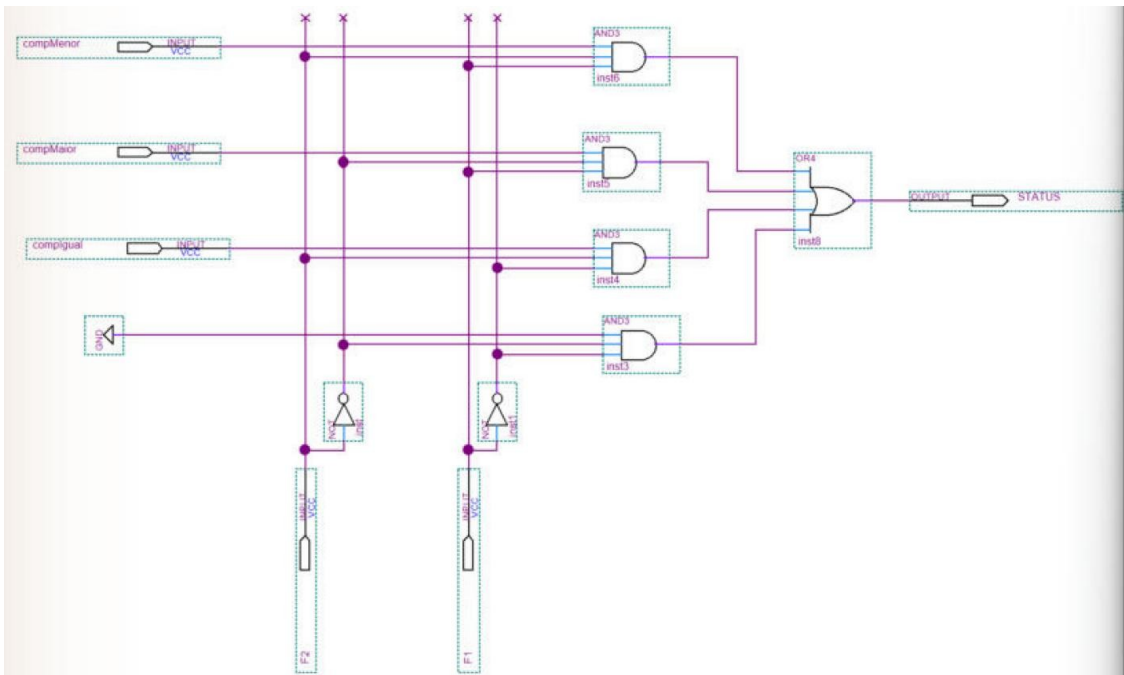


Figura 33 – Circuito do mux_comparadores (lib/mux_comparadores.bdf)

Simulação

[inserir figura: testes/mux_comparadores/mux_comparadores_simulacao.png]

Figura 34 – Simulação do mux_comparadores (mux_comparadores.vwf)

4.6 Seleção da saída

4.6.1 decodificador_saida

Funcionamento

Traduz o código da operação no seletor do mux_saida, que escolhe entre soma/subtração (00), C2 de B (01), AND (10) e XOR (11). Nas comparações (011, 100, 101) a saída não

importa (X), porque F é zerado pelo sinal F_EN da ula; esses X foram usados para simplificar. Entre parênteses, o valor que o circuito gera nesses casos.

- **Entradas:** S3, S2, S1 (= S[2], S[1], S[0])
- **Saídas:** F1, F2 (F1 → S[1] e F2 → S[0] do mux_saida)

Tabela verdade

Tabela 32 – decodificador_saida: tabela verdade

S3	S2	S1	Operação	F1	F2	mux_saida escolhe
0	0	0	A + B	0	0	soma/subtração
0	0	1	A – B	0	0	soma/subtração
0	1	0	C2 de B	0	1	C2 de B
0	1	1	A = B	X (0)	X (1)	— (F zerado por F_EN)
1	0	0	A > B	X (1)	X (0)	— (F zerado por F_EN)
1	0	1	A < B	X (1)	X (0)	— (F zerado por F_EN)
1	1	0	A AND B	1	0	AND
1	1	1	A XOR B	1	1	XOR

Equações

- $F1 = S3$
- $F2 = S3' \cdot S2 + S2 \cdot S1 = S2 \cdot (S3' + S1)$

Mapas de Karnaugh

Tabela 33 – decodificador_saida: mapa de Karnaugh de F1

S3 \ S2S1	00	01	11	10
0	0	0	X	0
1	X	X	1	1

Usando os X, a linha S3 = 1 inteira vira um grupo de quatro: $F1 = S3$.

Tabela 34 – decodificador_saida: mapa de Karnaugh de F2

S3 \ S2S1	00	01	11	10
0	0	0	X	1
1	X	X	1	0

Dois grupos de dois (com X): $S3' \cdot S2$ e $S2 \cdot S1$.

Circuito

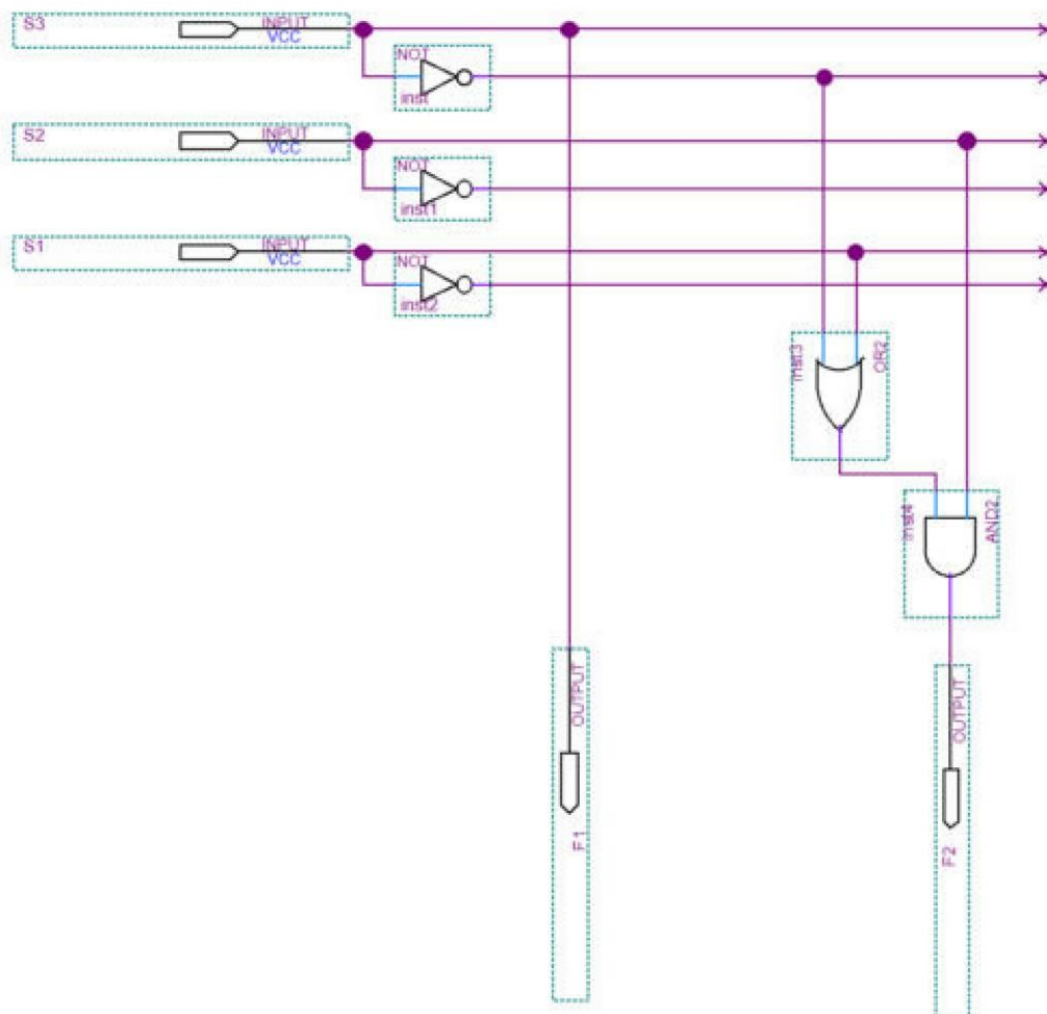


Figura 35 – Circuito do decodificador_saida (lib/decodificador_saida.bdf)

Simulação

[inserir figura: testes/decodificador_saida/decodificador_saida_simulacao.png]

Figura 36 – Simulação do decodificador_saida (decodificador_saida.vwf)

4.6.2 mux_saida

Funcionamento

Multiplexador 4:1 de 6 bits: seis mux4x1, um por bit, com o mesmo seletor S[1..0] (vindo do decodificador_saida). Escolhe qual dos quatro resultados vetoriais vai para F.

- **Usa:** mux4x1 ×6
- **Entradas:** SOMA_OU_SUB[5..0], Comp2B[5..0], OP_AND[5..0], OP_XOR[5..0], S[1..0]
- **Saídas:** F[5..0]

Tabela verdade

Tabela 35 – mux_saida: tabela verdade

S[1]	S[0]	F[5..0]	Operação
0	0	SOMA_OU_SUB[5..0]	000, 001
0	1	Comp2B[5..0]	010
1	0	OP_AND[5..0]	110
1	1	OP_XOR[5..0]	111

Equações

Para cada bit $k = 0 \dots 5$:

- $$F[k] = \text{SOMA_OU_SUB}[k] \cdot S1' \cdot S0' + \text{Comp2B}[k] \cdot S1' \cdot S0 + \text{OP_AND}[k] \cdot S1 \cdot S0' + \text{OP_XOR}[k] \cdot S1 \cdot S0$$

Circuito

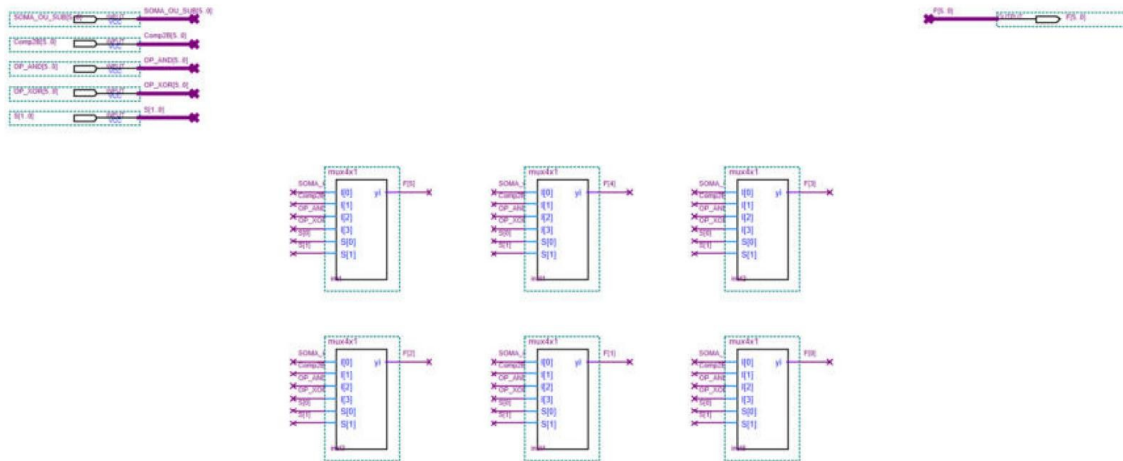


Figura 37 – Circuito do mux_saida (lib/mux_saida.bdf)

Simulação

[inserir figura: testes/mux_saida/mux_saida_simulacao.png]

Figura 38 – Simulação do mux_saida (mux_saida.vwf)

4.7 Displays

4.7.1 decod7seg_ab_dezena

Funcionamento

Gera os segmentos da dezena de uma magnitude de 0 a 15 ($|A|$ no HEX7 e $|B|$ no HEX5). A dezena é 1 quando o valor é ≥ 10 e 0 nos outros casos. Os segmentos b e c acendem nos dois dígitos (sempre 0), g nunca acende (sempre 1), e a, d, e, f só apagam quando o dígito é 1.

- Entradas:** A, B, C, D (A = bit mais significativo, peso 8)

- **Saídas:** seg_a, seg_b, seg_c, seg_d, seg_e, seg_f, seg_g (ativo em nível baixo: 0 acende)

Tabela verdade

Tabela 36 – decod7seg_ab_dezena: tabela verdade

A	B	C	D	Valor	Dígito	seg_a	seg_b	seg_c	seg_d	seg_e	seg_f	seg_g
0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	0	0	0	0	0	1
0	0	1	0	2	0	0	0	0	0	0	0	1
0	0	1	1	3	0	0	0	0	0	0	0	1
0	1	0	0	4	0	0	0	0	0	0	0	1
0	1	0	1	5	0	0	0	0	0	0	0	1
0	1	1	0	6	0	0	0	0	0	0	0	1
0	1	1	1	7	0	0	0	0	0	0	0	1
1	0	0	0	8	0	0	0	0	0	0	0	1
1	0	0	1	9	0	0	0	0	0	0	0	1
1	0	1	0	10	1	1	0	0	1	1	1	1
1	0	1	1	11	1	1	0	0	1	1	1	1
1	1	0	0	12	1	1	0	0	1	1	1	1
1	1	0	1	13	1	1	0	0	1	1	1	1
1	1	1	0	14	1	1	0	0	1	1	1	1
1	1	1	1	15	1	1	0	0	1	1	1	1

Equações

Equações implementadas no circuito (' = NOT; cada termo é um grupo do mapa-K):

- $\text{seg_a} = AB + AC$
- $\text{seg_b} = 0$
- $\text{seg_c} = 0$
- $\text{seg_d} = AB + AC$
- $\text{seg_e} = AB + AC$
- $\text{seg_f} = AB + AC$
- $\text{seg_g} = 1$

Mapas de Karnaugh

Linhas AB, colunas CD. Os segmentos a, d, e e f têm o mesmo mapa: 1 de 10 a 15, ou seja, $A \cdot B + A \cdot C = A \cdot (B + C)$.

Tabela 37 – decod7seg_ab_dezena: mapa de Karnaugh de seg_a

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	0	1	1

$\text{seg_b} = 0$ (constante)

$\text{seg_c} = 0$ (constante)

seg_d : mesmo mapa de seg_a .

seg_e : mesmo mapa de seg_a .

seg_f: mesmo mapa de **seg_a**.

seg_g = 1 (constante)

Circuito

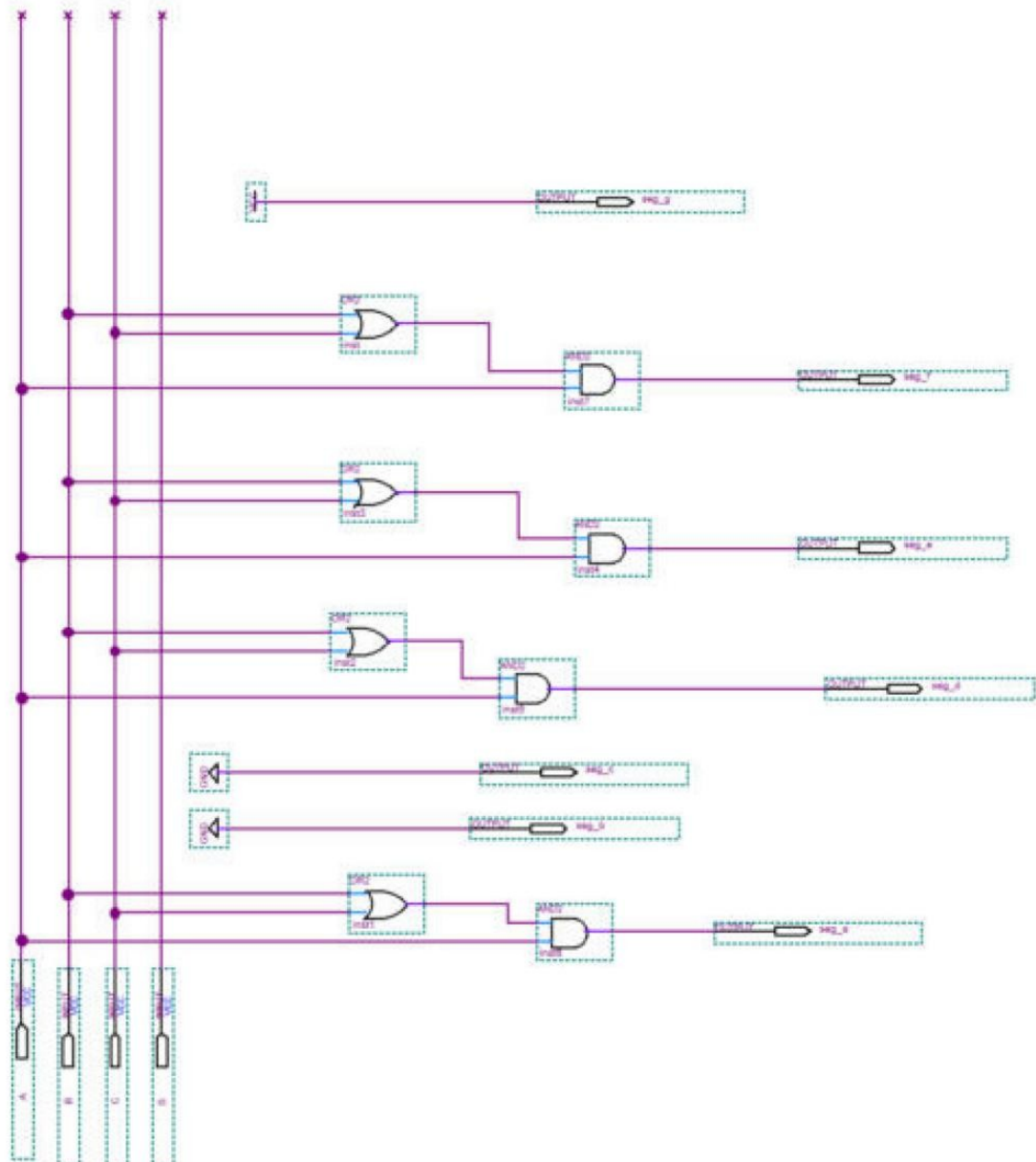


Figura 39 – Circuito do decod7seg_ab_dezena (lib/decod7seg_ab_dezena.bdf)

Simulação

[inserir figura: testes/decod7seg_ab_dezena/decod7seg_ab_dezena_simulacao.png]

Figura 40 – Simulação do decod7seg_ab_dezena (decod7seg_ab_dezena.vwf)

4.7.2 decod7seg_ab_unidade

Funcionamento

Gera os segmentos da unidade de uma magnitude de 0 a 15 (|A| no HEX6 e |B| no HEX4): o dígito é o valor módulo 10. Em vez de converter para BCD, cada segmento é uma soma de produtos tirada direto da tabela abaixo.

- **Entradas:** A, B, C, D (A = bit mais significativo, peso 8)
- **Saídas:** au_seg, bu_seg, cu_seg, du_seg, eu_seg, fu_seg, gu_seg (ativo em nível baixo: 0 acende)

Tabela verdade

Tabela 38 – decod7seg_ab_unidade: tabela verdade

A	B	C	D	Valor	Dígito	au_seg	bu_seg	cu_seg	du_seg	eu_seg	fu_seg	gu_seg
0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	1	1	1	0	0	1	1	1	1
0	0	1	0	2	2	0	0	1	0	0	1	0
0	0	1	1	3	3	0	0	0	0	1	1	0
0	1	0	0	4	4	1	0	0	1	1	0	0
0	1	0	1	5	5	0	1	0	0	1	0	0
0	1	1	0	6	6	0	1	0	0	0	0	0
0	1	1	1	7	7	0	0	0	1	1	1	1
1	0	0	0	8	8	0	0	0	0	0	0	0
1	0	0	1	9	9	0	0	0	0	1	0	0
1	0	1	0	10	0	0	0	0	0	0	0	1
1	0	1	1	11	1	1	0	0	1	1	1	1
1	1	0	0	12	2	0	0	1	0	0	1	0
1	1	0	1	13	3	0	0	0	0	1	1	0
1	1	1	0	14	4	1	0	0	1	1	0	0
1	1	1	1	15	5	0	1	0	0	1	0	0

Equações

Equações implementadas no circuito (' = NOT; cada termo é um grupo do mapa-K):

- $au_seg = A'B'C'D + A'BC'D' + AB'CD + ABCD'$
- $bu_seg = ABCD + A'BCD' + A'BC'D$
- $cu_seg = A'B'CD' + ABC'D'$
- $du_seg = AB'CD + A'BC'D' + A'B'C'D + A'BCD + ABCD'$
- $eu_seg = D + A'BC' + ABC$
- $fu_seg = B'CD + A'B'D + A'B'C + A'CD + ABC'$
- $gu_seg = AB'C + A'B'C' + A'BCD$

Mapas de Karnaugh

Linhas AB, colunas CD. Cada mapa marca com 1 os valores em que o segmento fica apagado.

Tabela 39 – decod7seg_ab_unidade: mapa de Karnaugh de au_seg

AB \ CD	00	01	11	10
00	0	1	0	0
01	1	0	0	0
11	0	0	0	1

AB \ CD	00	01	11	10
10	0	0	1	0

Tabela 40 – decod7seg_ab_unidade: mapa de Karnaugh de bu_seg

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	0	1
11	0	0	1	0
10	0	0	0	0

Tabela 41 – decod7seg_ab_unidade: mapa de Karnaugh de cu_seg

AB \ CD	00	01	11	10
00	0	0	0	1
01	0	0	0	0
11	1	0	0	0
10	0	0	0	0

Tabela 42 – decod7seg_ab_unidade: mapa de Karnaugh de du_seg

AB \ CD	00	01	11	10
00	0	1	0	0
01	1	0	1	0
11	0	0	0	1
10	0	0	1	0

Tabela 43 – decod7seg_ab_unidade: mapa de Karnaugh de eu_seg

AB \ CD	00	01	11	10
00	0	1	1	0
01	1	1	1	0
11	0	1	1	1
10	0	1	1	0

Tabela 44 – decod7seg_ab_unidade: mapa de Karnaugh de fu_seg

AB \ CD	00	01	11	10
00	0	1	1	1
01	0	0	1	0
11	1	1	0	0
10	0	0	1	0

Tabela 45 – decod7seg_ab_unidade: mapa de Karnaugh de gu_seg

AB \ CD	00	01	11	10
00	1	1	0	0
01	0	0	1	0
11	0	0	0	0
10	0	0	1	1

Circuito

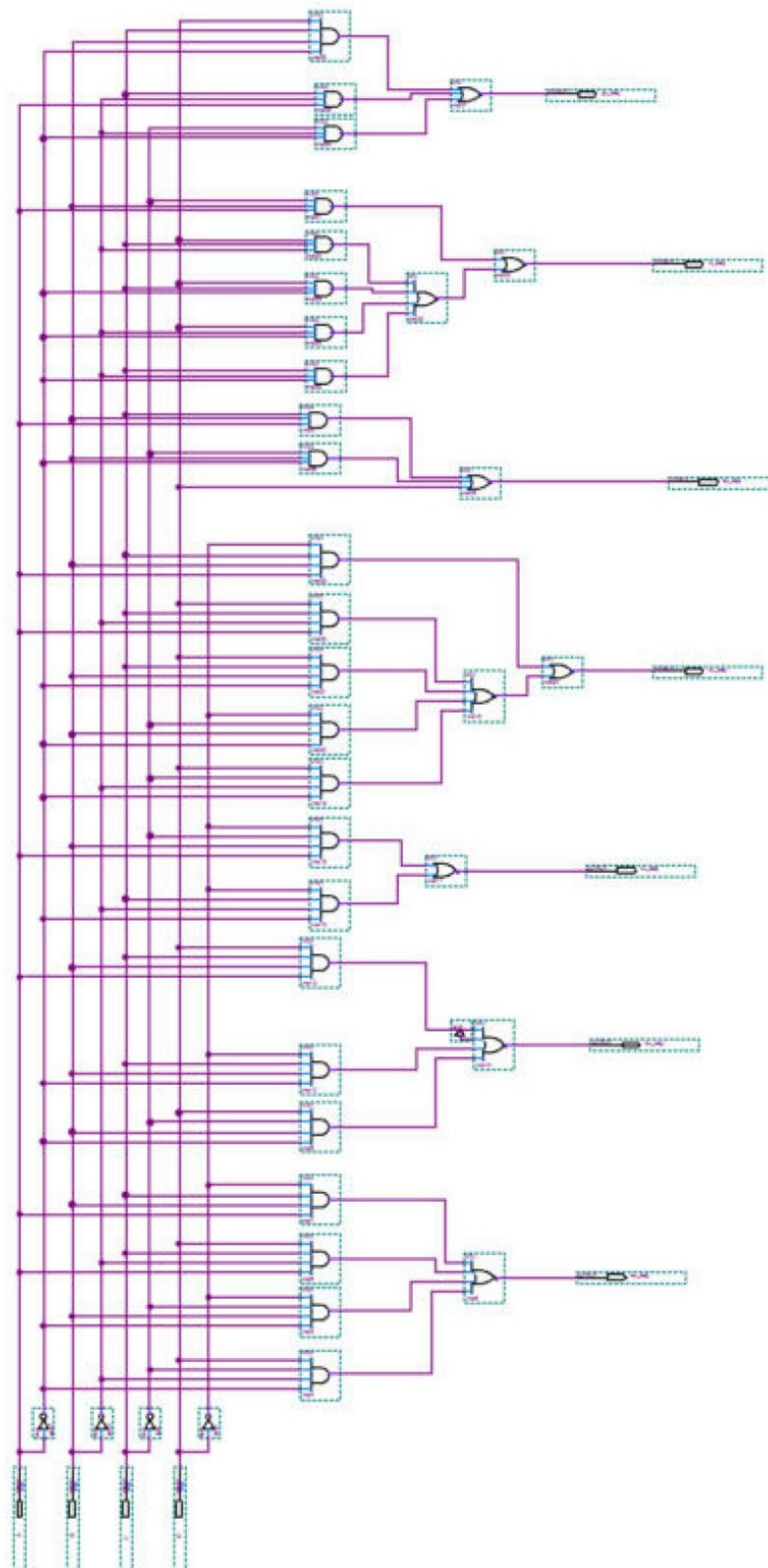


Figura 41 – Circuito do decod7seg_ab_unidade (lib/decod7seg_ab_unidade.bdf)

Simulação

[inserir figura: testes/decod7seg_ab_unidade/decod7seg_ab_unidade_simulacao.png]

Figura 42 – Simulação do decod7seg_ab_unidade (decod7seg_ab_unidade.vwf)

4.7.3 decod7seg_f_dezena

Funcionamento

Gera os segmentos da dezena de |F| (0 a 30), mostrada no HEX1: o dígito vai de 0 a 3. A entrada 31 não ocorre (X) e foi usada para simplificar.

- **Entradas:** R4 ... R0 (R4 = bit mais significativo, peso 16)
- **Saídas:** aDEZ, bDEZ, cDEZ, dDEZ, eDEZ, fDEZ, gDEZ (ativo em nível baixo: 0 acende)

Tabela verdade

Tabela 46 – decod7seg_f_dezena: tabela verdade

R4	R3	R2	R1	R0	Valor	Dígito	aDEZ	bDEZ	cDEZ	dDEZ	eDEZ	fDEZ	gDEZ
0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	1	1	0	0	0	0	0	0	0	1
0	0	0	1	0	2	0	0	0	0	0	0	0	1
0	0	0	1	1	3	0	0	0	0	0	0	0	1
0	0	1	0	0	4	0	0	0	0	0	0	0	1
0	0	1	0	1	5	0	0	0	0	0	0	0	1
0	0	1	1	0	6	0	0	0	0	0	0	0	1
0	0	1	1	1	7	0	0	0	0	0	0	0	1
0	1	0	0	0	8	0	0	0	0	0	0	0	1
0	1	0	0	1	9	0	0	0	0	0	0	0	1
0	1	0	1	0	10	1	1	0	0	1	1	1	1
0	1	0	1	1	11	1	1	0	0	1	1	1	1
0	1	1	0	0	12	1	1	0	0	1	1	1	1
0	1	1	0	1	13	1	1	0	0	1	1	1	1
0	1	1	1	0	14	1	1	0	0	1	1	1	1
0	1	1	1	1	15	1	1	0	0	1	1	1	1
1	0	0	0	0	16	1	1	0	0	1	1	1	1
1	0	0	0	1	17	1	1	0	0	1	1	1	1
1	0	0	1	0	18	1	1	0	0	1	1	1	1
1	0	0	1	1	19	1	1	0	0	1	1	1	1
1	0	1	0	0	20	2	0	0	1	0	0	1	0
1	0	1	0	1	21	2	0	0	1	0	0	1	0
1	0	1	1	0	22	2	0	0	1	0	0	1	0
1	0	1	1	1	23	2	0	0	1	0	0	1	0
1	1	0	0	0	24	2	0	0	1	0	0	1	0
1	1	0	0	1	25	2	0	0	1	0	0	1	0
1	1	0	1	0	26	2	0	0	1	0	0	1	0
1	1	0	1	1	27	2	0	0	1	0	0	1	0
1	1	1	0	0	28	2	0	0	1	0	0	1	0
1	1	1	0	1	29	2	0	0	1	0	0	1	0
1	1	1	1	0	30	3	0	0	0	0	1	1	0
1	1	1	1	1	31	—	X	X	X	X	X	X	X

Equações

Equações implementadas no circuito (' = NOT; cada termo é um grupo do mapa-K):

- $aDEZ = R4'R3R1 + R4'R3R2 + R4R3'R2'$
- $bDEZ = 0$
- $cDEZ = R4R3R2' + R4R3'R2 + R4R3R1'$

- $dDEZ = R4'R3R1 + R4'R3R2 + R4R3'R2'$
- $eDEZ = R3R2R1 + R4'R3R1 + R4'R3R2 + R4R3'R2'$
- $fDEZ = R4 + R3R2 + R3R1$
- $gDEZ = R4' + R3'R2'$

Mapas de Karnaugh

Cinco variáveis: um mapa para $R4 = 0$ e outro para $R4 = 1$; linhas $R3R2$, colunas $R1R0$. A dezena só depende de $R4$, $R3$, $R2$ e $R1$, porque $R0$ não muda a dezena.

Tabela 47 – decod7seg_f_dezena: mapa de Karnaugh de aDEZ, $R4 = 0$

$R3R2 \setminus R1R0$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	0	1	1

Tabela 48 – decod7seg_f_dezena: mapa de Karnaugh de aDEZ, $R4 = 1$

$R3R2 \setminus R1R0$	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	X	0
10	0	0	0	0

bDEZ = 0 (constante)

Tabela 49 – decod7seg_f_dezena: mapa de Karnaugh de cDEZ, $R4 = 0$

$R3R2 \setminus R1R0$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

Tabela 50 – decod7seg_f_dezena: mapa de Karnaugh de cDEZ, $R4 = 1$

$R3R2 \setminus R1R0$	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	1	X	0
10	1	1	1	1

dDEZ: mesmo mapa de **aDEZ**.

Tabela 51 – decod7seg_f_dezena: mapa de Karnaugh de eDEZ, $R4 = 0$

$R3R2 \setminus R1R0$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	0	1	1

Tabela 52 – decod7seg_f_dezena: mapa de Karnaugh de eDEZ, R4 = 1

R3R2 \ R1R0	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	X	1
10	0	0	0	0

Tabela 53 – decod7seg_f_dezena: mapa de Karnaugh de fDEZ, R4 = 0

R3R2 \ R1R0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	0	1	1

Tabela 54 – decod7seg_f_dezena: mapa de Karnaugh de fDEZ, R4 = 1

R3R2 \ R1R0	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	X	1
10	1	1	1	1

Tabela 55 – decod7seg_f_dezena: mapa de Karnaugh de gDEZ, R4 = 0

R3R2 \ R1R0	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

Tabela 56 – decod7seg_f_dezena: mapa de Karnaugh de gDEZ, R4 = 1

R3R2 \ R1R0	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	X	0
10	0	0	0	0

Circuito

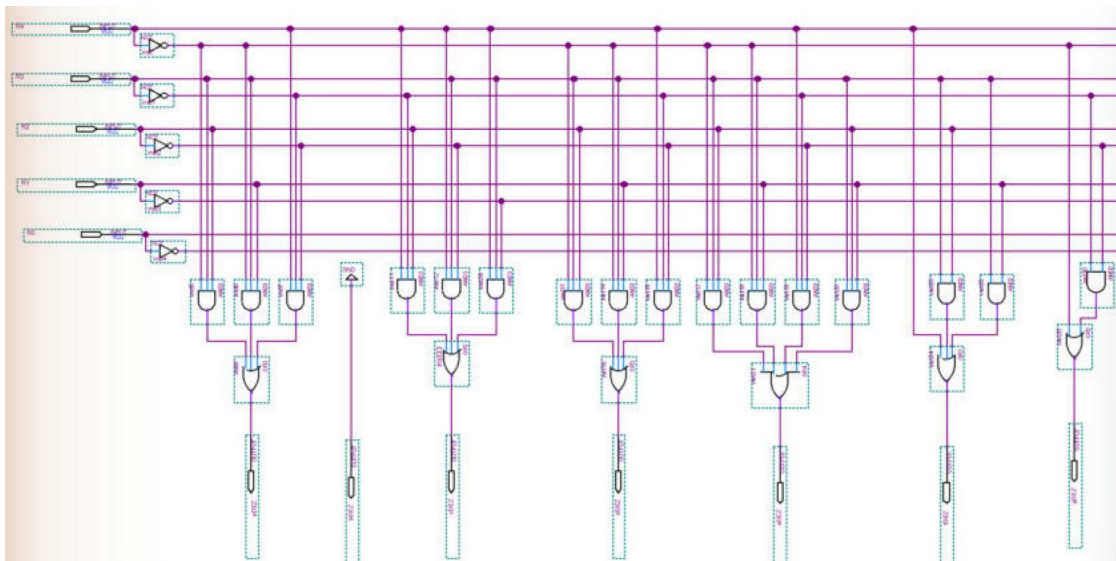


Figura 43 – Circuito do decod7seg_f_dezena (lib/decod7seg_f_dezena.bdf)

Simulação

[inserir figura: testes/decod7seg_f_dezena/decod7seg_f_dezena_simulacao.png]

Figura 44 – Simulação do decod7seg_f_dezena (decod7seg_f_dezena.vwf)

4.7.4 decod7seg_f_unidade

Funcionamento

Gera os segmentos da unidade de $|F|$ (0 a 30), mostrada no HEX0: o dígito é o valor módulo 10. A entrada 31 não ocorre (X).

- **Entradas:** R4 ... R0 (R4 = bit mais significativo, peso 16)
- **Saídas:** aUNI, bUNI, cUNI, dUNI, eUNI, fUNI, gUNI (ativo em nível baixo: 0 acende)

Tabela verdade

Tabela 57 – decod7seg_f_unidade: tabela verdade

R4	R3	R2	R1	R0	Valor	Dígito	aUNI	bUNI	cUNI	dUNI	eUNI	fUNI	gUNI
0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	1	1	1	1	0	0	1	1	1	1
0	0	0	1	0	2	2	0	0	1	0	0	1	0
0	0	0	1	1	3	3	0	0	0	0	1	1	0
0	0	1	0	0	4	4	1	0	0	1	1	0	0
0	0	1	0	1	5	5	0	1	0	0	1	0	0
0	0	1	1	0	6	6	0	1	0	0	0	0	0
0	0	1	1	1	7	7	0	0	0	1	1	1	1
0	1	0	0	0	8	8	0	0	0	0	0	0	0
0	1	0	0	1	9	9	0	0	0	0	1	0	0
0	1	0	1	0	10	0	0	0	0	0	0	0	1
0	1	0	1	1	11	1	1	0	0	1	1	1	1
0	1	1	0	0	12	2	0	0	1	0	0	1	0
0	1	1	0	1	13	3	0	0	0	0	1	1	0

R4	R3	R2	R1	R0	Valor	Dígito	aUNI	bUNI	cUNI	dUNI	eUNI	fUNI	gUNI
0	1	1	1	0	14	4	1	0	0	1	1	0	0
0	1	1	1	1	15	5	0	1	0	0	1	0	0
1	0	0	0	0	16	6	0	1	0	0	0	0	0
1	0	0	0	1	17	7	0	0	0	1	1	1	1
1	0	0	1	0	18	8	0	0	0	0	0	0	0
1	0	0	1	1	19	9	0	0	0	0	1	0	0
1	0	1	0	0	20	0	0	0	0	0	0	0	1
1	0	1	0	1	21	1	1	0	0	1	1	1	1
1	0	1	1	0	22	2	0	0	1	0	0	1	0
1	0	1	1	1	23	3	0	0	0	0	1	1	0
1	1	0	0	0	24	4	1	0	0	1	1	0	0
1	1	0	0	1	25	5	0	1	0	0	1	0	0
1	1	0	1	0	26	6	0	1	0	0	0	0	0
1	1	0	1	1	27	7	0	0	0	1	1	1	1
1	1	1	0	0	28	8	0	0	0	0	0	0	0
1	1	1	0	1	29	9	0	0	0	0	1	0	0
1	1	1	1	0	30	0	0	0	0	0	0	0	1
1	1	1	1	1	31	—	X	X	X	X	X	X	X

Equações

Equações implementadas no circuito (' = NOT; cada termo é um grupo do mapa-K):

- $aUNI = R4R3R2'R1'R0' + R4'R3R2R1R0' + R4R3'R2R1'R0 + R4'R3R2'R1R0 + R4'R3'R2R1'R0' + R4'R3'R2'R1'R0 + R4R3R2R1R0$
- $bUNI = R4R3R2'R1R0' + R4R3'R2'R1'R0' + R4R3R2'R1'R0 + R4'R3R2R1R0 + R4'R3'R2R1R0' + R4'R3'R2R1'R0$
- $cUNI = R4'R3R2R1'R0' + R4'R3'R2'R1R0' + R4R3'R2R1R0'$
- $dUNI = R4R3R2'R1'R0' + R4'R3'R2R1R0 + R4'R3R2R1R0' + R4'R3'R2R1'R0' + R4R3'R1'R0 + R4R3R1R0 + R3R2'R1R0 + R3'R2'R1'R0$
- $eUNI = R0 + R4R3R2'R1' + R4'R3'R2R1' + R4'R3R2R1$
- $fUNI = R4R2R1R0 + R4R3'R1'R0 + R4R3'R2R1 + R4'R3R2R1' + R4'R3'R1R0 + R3R2'R1R0 + R4'R3'R2'R1 + R4'R3'R2'R0$
- $gUNI = R4R3R2R1 + R3R2'R1R0 + R4R3'R2R1' + R4'R3R2'R1 + R3'R2'R1'R0 + R4'R3'R2'R1' + R4'R3'R2R1R0$

Mapas de Karnaugh

Cinco variáveis: um mapa para $R4 = 0$ e outro para $R4 = 1$; linhas $R3R2$, colunas $R1R0$. Como os 1s se repetem de 10 em 10, poucos ficam adjacentes, e várias equações têm termos com as cinco variáveis.

Tabela 58 – decod7seg_f_unidade: mapa de Karnaugh de aUNI, $R4 = 0$

R3R2 \ R1R0	00	01	11	10
00	0	1	0	0
01	1	0	0	0
11	0	0	0	1
10	0	0	1	0

Tabela 59 – decod7seg_f_unidade: mapa de Karnaugh de aUNI, $R4 = 1$

R3R2 \ R1R0	00	01	11	10
00	0	0	0	0
01	0	1	0	0

R3R2 \ R1R0	00	01	11	10
11	0	0	X	0
10	1	0	0	0

Tabela 60 – decod7seg_f_unidade: mapa de Karnaugh de bUNI, R4 = 0

R3R2 \ R1R0	00	01	11	10
00	0	0	0	0
01	0	1	0	1
11	0	0	1	0
10	0	0	0	0

Tabela 61 – decod7seg_f_unidade: mapa de Karnaugh de bUNI, R4 = 1

R3R2 \ R1R0	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	0	0	X	0
10	0	1	0	1

Tabela 62 – decod7seg_f_unidade: mapa de Karnaugh de cUNI, R4 = 0

R3R2 \ R1R0	00	01	11	10
00	0	0	0	1
01	0	0	0	0
11	1	0	0	0
10	0	0	0	0

Tabela 63 – decod7seg_f_unidade: mapa de Karnaugh de cUNI, R4 = 1

R3R2 \ R1R0	00	01	11	10
00	0	0	0	0
01	0	0	0	1
11	0	0	X	0
10	0	0	0	0

Tabela 64 – decod7seg_f_unidade: mapa de Karnaugh de dUNI, R4 = 0

R3R2 \ R1R0	00	01	11	10
00	0	1	0	0
01	1	0	1	0
11	0	0	0	1
10	0	0	1	0

Tabela 65 – decod7seg_f_unidade: mapa de Karnaugh de dUNI, R4 = 1

R3R2 \ R1R0	00	01	11	10
00	0	1	0	0
01	0	1	0	0
11	0	0	X	0
10	1	0	1	0

Tabela 66 – decod7seg_f_unidade: mapa de Karnaugh de eUNI, R4 = 0

R3R2 \ R1R0	00	01	11	10
00	0	1	1	0
01	1	1	1	0

R3R2 \ R1R0	00	01	11	10
11	0	1	1	1
10	0	1	1	0

Tabela 67 – decod7seg_f_unidade: mapa de Karnaugh de eUNI, R4 = 1

R3R2 \ R1R0	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	X	0
10	1	1	1	0

Tabela 68 – decod7seg_f_unidade: mapa de Karnaugh de fUNI, R4 = 0

R3R2 \ R1R0	00	01	11	10
00	0	1	1	1
01	0	0	1	0
11	1	1	0	0
10	0	0	1	0

Tabela 69 – decod7seg_f_unidade: mapa de Karnaugh de fUNI, R4 = 1

R3R2 \ R1R0	00	01	11	10
00	0	1	0	0
01	0	1	1	1
11	0	0	X	0
10	0	0	1	0

Tabela 70 – decod7seg_f_unidade: mapa de Karnaugh de gUNI, R4 = 0

R3R2 \ R1R0	00	01	11	10
00	1	1	0	0
01	0	0	1	0
11	0	0	0	0
10	0	0	1	1

Tabela 71 – decod7seg_f_unidade: mapa de Karnaugh de gUNI, R4 = 1

R3R2 \ R1R0	00	01	11	10
00	0	1	0	0
01	1	1	0	0
11	0	0	X	1
10	0	0	1	0

Circuito

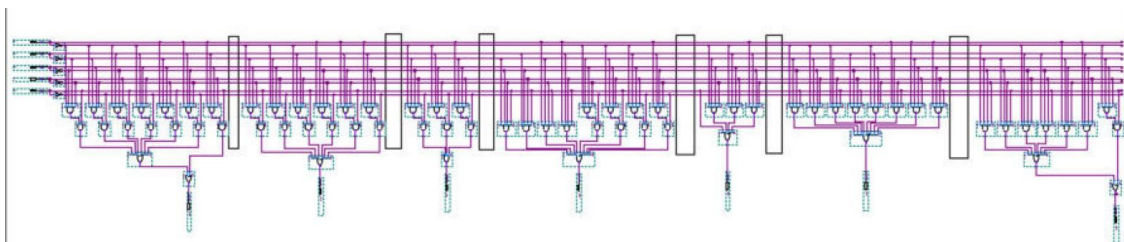


Figura 45 – Circuito do decod7seg_f_unidade (lib/decod7seg_f_unidade.bdf)

Simulação

[inserir figura: testes/decod7seg_f_unidade/decod7seg_f_unidade_simulacao.png]

Figura 46 – Simulação do decod7seg_f_unidade (decod7seg_f_unidade.vwf)

4.7.5 decod7seg_f

Funcionamento

Junta os decodificadores de dezena e unidade de |F|. O sinal de F é tratado à parte, como pede o enunciado: `led_negativo = sinal_S`. No toplevel, as saídas passam pelo `apaga_display` antes de chegar ao HEX1 e ao HEX0.

- **Usa:** `decod7seg_f_dezena`, `decod7seg_f_unidade`
- **Entradas:** `S[4..0]` (|F|, 0 a 30), `sinal_S`
- **Saídas:** `aDEZ ... gDEZ`, `aUNI ... gUNI`, `led_negativo`

Tabela verdade

Tabela 72 – decod7seg_f: tabela verdade

S[4..0]		F		Dezena	aDEZ... gDEZ
00000	0	0	0000001	0	0000001
00001	1	0	0000001	1	1001111
00010	2	0	0000001	2	0010010
00011	3	0	0000001	3	0000110
00100	4	0	0000001	4	1001100
00101	5	0	0000001	5	0100100
00110	6	0	0000001	6	0100000
00111	7	0	0000001	7	0001111
01000	8	0	0000001	8	0000000
01001	9	0	0000001	9	0000100
01010	10	1	1001111	0	0000001
01011	11	1	1001111	1	1001111
01100	12	1	1001111	2	0010010
01101	13	1	1001111	3	0000110
01110	14	1	1001111	4	1001100
01111	15	1	1001111	5	0100100
10000	16	1	1001111	6	0100000
10001	17	1	1001111	7	0001111
10010	18	1	1001111	8	0000000
10011	19	1	1001111	9	0000100
10100	20	2	0010010	0	0000001
10101	21	2	0010010	1	1001111
10110	22	2	0010010	2	0010010
10111	23	2	0010010	3	0000110
11000	24	2	0010010	4	1001100
11001	25	2	0010010	5	0100100

S[4..0]		F		Dezena	aDEZ... gDEZ
11010	26	2	0010010	6	0100000
11011	27	2	0010010	7	0001111
11100	28	2	0010010	8	0000000
11101	29	2	0010010	9	0000100
11110	30	3	0000110	0	0000001

Equações

- `led_negativo = sinal_S`
- `dezena e unidade: ver decod7seg_f_dezena e decod7seg_f_unidade`

Circuito

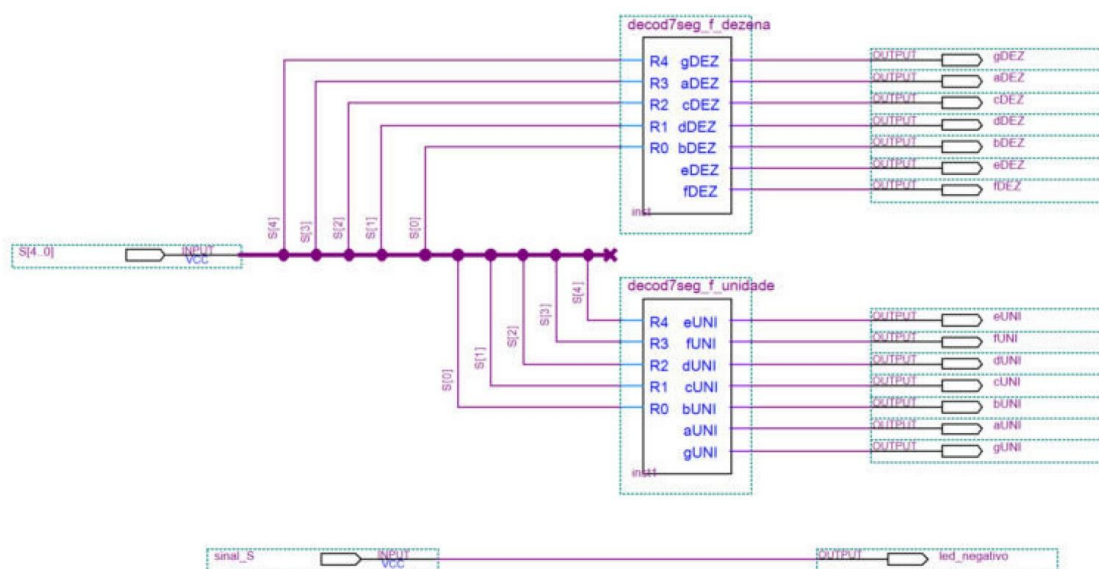


Figura 47 – Circuito do decod7seg_f (lib/decod7seg_f.bdf)

Simulação

[inserir figura: testes/decod7seg_f/decod7seg_f_simulacao.png]

Figura 48 – Simulação do decod7seg_f (decod7seg_f.vwf)

4.7.6 apaga_display

Funcionamento

Apaga um display quando $EN = 0$. Como os segmentos acendem com 0, basta forçar todos em 1: cada segmento passa por uma porta OR com EN' . Com $EN = 1$, os segmentos passam sem alteração. Na placa, $EN = DISP_EN$, então os displays de F só acendem na soma e na subtração.

- **Entradas:** $I[6..0]$ (segmentos; $I[0] = a \dots I[6] = g$), EN
- **Saídas:** $O[6..0]$

Tabela verdade

Tabela 73 – `apaga_display`: tabela verdade (um segmento)

EN	I[k]	O[k]
0	0	1
0	1	1
1	0	0
1	1	1

Tabela 74 – `apaga_display`: tabela verdade (barramento)

EN	O[6..0]
0	1111111 (apagado)
1	I[6..0]

Equações

- $O[k] = I[k] + EN'$, para $k = 0 \dots 6$

Circuito

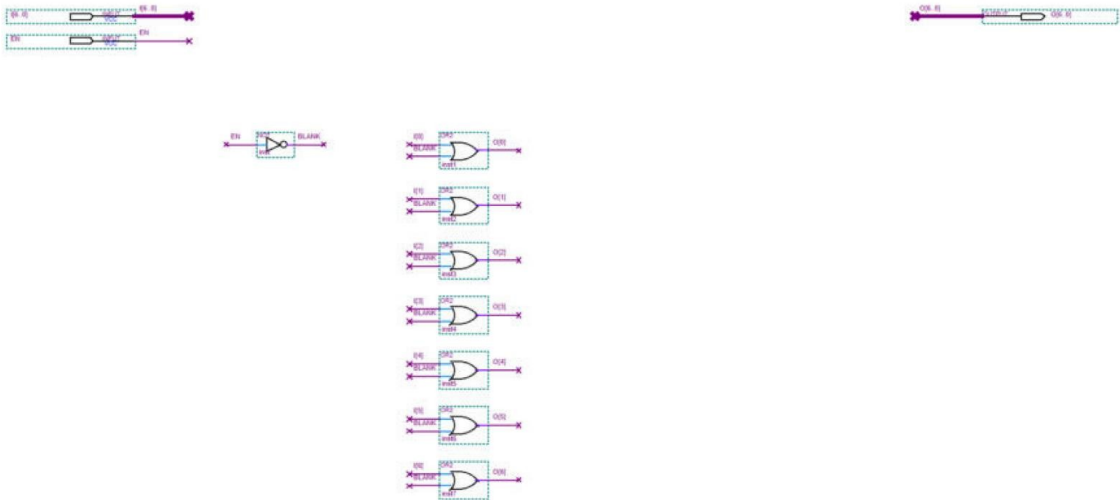


Figura 49 – Circuito do `apaga_display` (lib/apaga_display.bdf)

Simulação

[inserir figura: testes/apaga_display/apaga_display_simulacao.png]

Figura 50 – Simulação do `apaga_display` (apaga_display.vwf)

5 Sistema completo

5.1 ULA

Funcionamento

Liga todos os blocos: somador_subtrator (soma e subtração), comp2 (C2 de B), op_and, op_xor, comparador_igual e duas instâncias do comp_maior (comparações), decodificador_saida + mux_saida (escolhem F) e decodificador_comparadores + mux_comparadores (escolhem STATUS). Acrescenta dois sinais de controle: F_EN, que zera F nas comparações (seis portas AND), e DISP_EN, que só deixa os displays de F acesos na soma e na subtração. A tabela completa tem 8192 linhas (2^{13}); abaixo, o comportamento por operação e casos de teste, que também foram conferidos na placa.

- **Usa:** todos os blocos da ULA
- **Entradas:** A[4..0], B[4..0] (sinal-magnitude, bit 4 = sinal), S[2..0]
- **Saídas:** F[5..0] (sinal-magnitude, F[5] = sinal), STATUS, DISP_EN

Tabela verdade

Tabela 75 – ula: tabela verdade (por operação)

S	Operação	F	STATUS	F_EN	DISP_EN
000	A + B	somador_subtrator	0	1	1
001	A – B	somador_subtrator	0	1	1
010	C2 de B	{0S, 0S, 0[3..0]} do comp2	0	1	0
011	A = B	000000	comparador_igual	0	0
100	A > B	000000	comp_maior(A, B)	0	0
101	A < B	000000	comp_maior(B, A)	0	0
110	A AND B	op_and	0	1	0
111	A XOR B	op_xor	0	1	0

Tabela 76 – ula: tabela verdade (casos de teste)

S	A	B	F[5..0]	STATUS	DISP_EN
000	00101 (+5)	10011 (-3)	000010	0	1
001	00010 (+2)	00101 (+5)	100011	0	1
001	11001 (-9)	01100 (+12)	110101	0	1
000	11000 (-8)	11000 (-8)	110000	0	1
000	11111 (-15)	11111 (-15)	111110	0	1
000	01111 (+15)	01111 (+15)	011110	0	1
001	00111 (+7)	00111 (+7)	000000	0	1
000	10000 (-0)	00000 (+0)	000000	0	1
010	00000 (+0)	10011 (-3)	111101	0	0
010	00000 (+0)	00101 (+5)	000101	0	0
010	00000 (+0)	11000 (-8)	111000	0	0

S	A	B	F[5..0]	STATUS	DISP_EN
011	10000 (-0)	00000 (+0)	000000	1	0
011	00011 (+3)	10011 (-3)	000000	0	0
100	00010 (+2)	10101 (-5)	000000	1	0
100	10010 (-2)	10101 (-5)	000000	1	0
101	10011 (-3)	10011 (-3)	000000	0	0
101	10111 (-7)	00001 (+1)	000000	1	0
110	00101 (+5)	00011 (+3)	000001	0	0
110	10101 (-5)	10011 (-3)	100001	0	0
111	00101 (+5)	00011 (+3)	000110	0	0
111	11010 (-10)	00110 (+6)	101100	0	0

Equações

- $F_EN = (S2 \odot S1) + S2' \cdot S0'$ e $F[k] = FM[k] \cdot F_EN$ (FM = saída do mux_saida)
- $DISP_EN = S2' \cdot S1'$

Mapas de Karnaugh

Tabela 77 – u1a: mapa de Karnaugh de F_EN

S2 \ S1S0	00	01	11	10
0	1	1	0	1
1	0	0	1	1

Grupos: $S2' \cdot S1'$, $S2 \cdot S1$ (que juntos formam $S2 \odot S1$) e $S2' \cdot S0'$.

Tabela 78 – u1a: mapa de Karnaugh de DISP_EN

S2 \ S1S0	00	01	11	10
0	1	1	0	0
1	0	0	0	0

Um grupo de dois: $S2' \cdot S1'$.

Circuito

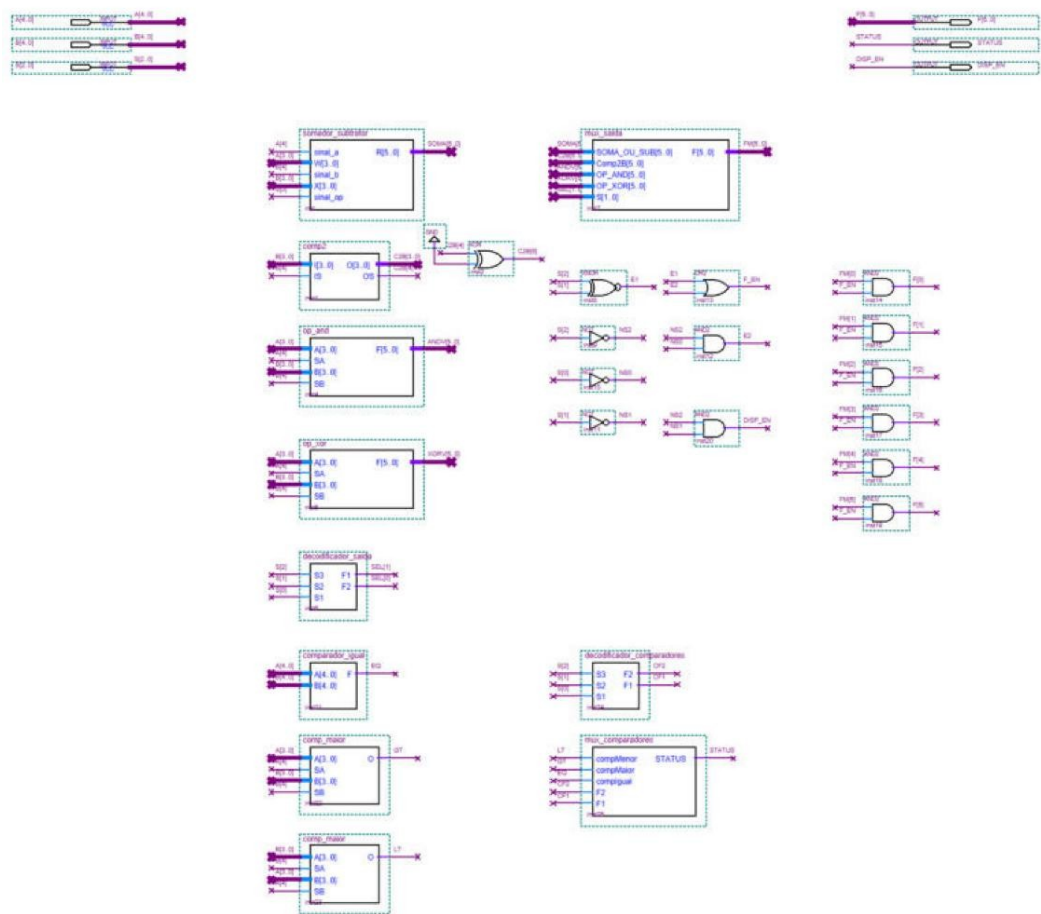


Figura 51 – Circuito do ula (lib/ula.bdf)

Simulação

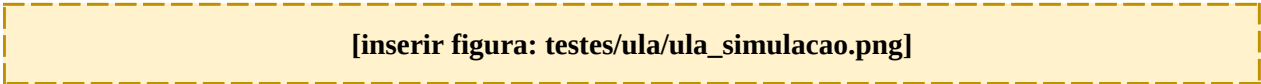


Figura 52 – Simulação do ula (ula.vwf)

5.2 Toplevel

O toplevel (src/toplevel.bdf) é o circuito que vai para a placa: liga a ula, os decodificadores e os LEDs aos pinos da DE2-115.

Tabela 79 – toplevel

De	Para
SW[17..13], SW[12..8], SW[2..0]	ula: A, B e S
SW[17..0]	LEDR[17..0] (cópia das chaves)
ula.F[5..0]	LEDG[5..0]
ula.STATUS	LEDG8
SW[16..13] (A)	decod7seg_ab_dezена → HEX7 e decod7seg_ab_unidade → HEX6

De	Para
SW[11..8] (B)	decod7seg_ab_dezena → HEX5 e decod7seg_ab_unidade → HEX4
ula.F[4..0] e F[5]	decod7seg_f → apaga_display (EN = DISP_EN) → HEX1 (dezena) e HEX0 (unidade)
NAND(F[5], DISP_EN)	HEX2[6] (segmento g, o “—”); HEX2[5..0] em VCC (apagados)

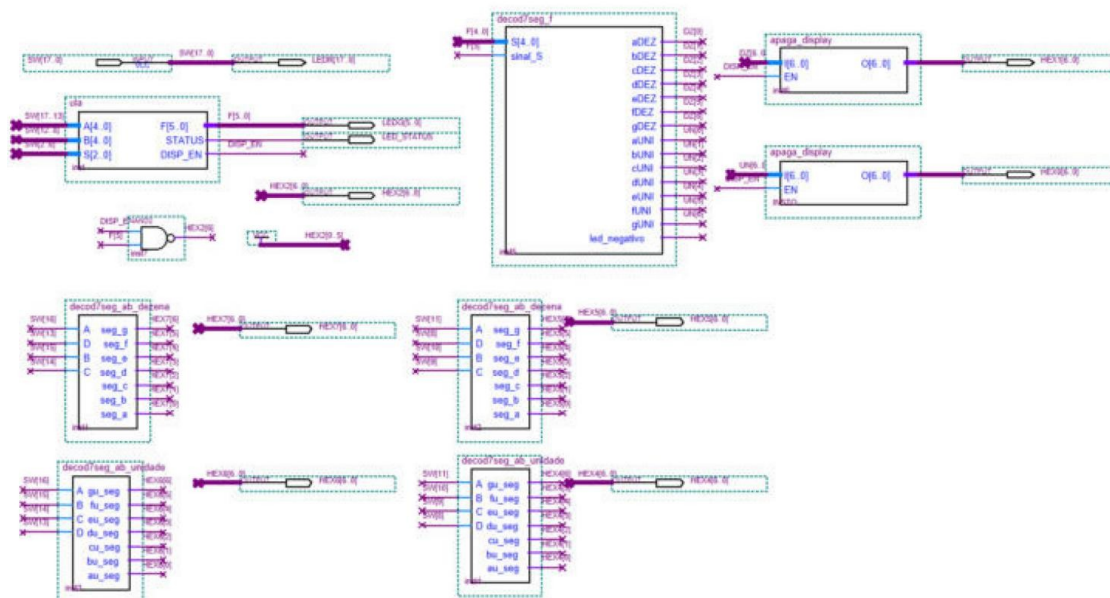


Figura 53 – Circuito do toplevel (src/toplevel.bdf)

5.3 Pinagem

A pinagem do FPGA Cyclone IV E EP4CE115F29C7 está em src/Projeto1SD.qsf (92 pinos). O ponto decimal dos displays não é ligado à FPGA na DE2-115; por isso os sinais de A e de B são mostrados nos LEDs LEDR17 e LEDR12.

Tabela 80 – pinagem (chaves)

Sinal	Pino	Função	Sinal	Pino	Função
SW[0]	PIN_AB28	S[0]	SW[9]	PIN_AB25	B[1]
SW[1]	PIN_AC28	S[1]	SW[10]	PIN_AC24	B[2]
SW[2]	PIN_AC27	S[2]	SW[11]	PIN_AB24	B[3]
SW[3]	PIN_AD27	—	SW[12]	PIN_AB23	sinal de B
SW[4]	PIN_AB27	—	SW[13]	PIN_AA24	A[0]
SW[5]	PIN_AC26	—	SW[14]	PIN_AA23	A[1]
SW[6]	PIN_AD26	—	SW[15]	PIN_AA22	A[2]
SW[7]	PIN_AB26	—	SW[16]	PIN_Y24	A[3]
SW[8]	PIN_AC25	B[0]	SW[17]	PIN_Y23	sinal de A

Tabela 81 – pinagem (LEDs)

Sinal	Pino	Sinal	Pino	Sinal	Pino
LEDR[0]	PIN_G19	LEDR[9]	PIN_G17	LEDG[0] (F0)	PIN_E21
LEDR[1]	PIN_F19	LEDR[10]	PIN_J15	LEDG[1] (F1)	PIN_E22
LEDR[2]	PIN_E19	LEDR[11]	PIN_H16	LEDG[2] (F2)	PIN_E25

Sinal	Pino	Sinal	Pino	Sinal	Pino
LEDR[3]	PIN_F21	LEDR[12]	PIN_J16	LEDG[3] (F3)	PIN_E24
LEDR[4]	PIN_F18	LEDR[13]	PIN_H17	LEDG[4] (F4)	PIN_H21
LEDR[5]	PIN_E18	LEDR[14]	PIN_F15	LEDG[5] (sinal de F)	PIN_G20
LEDR[6]	PIN_J19	LEDR[15]	PIN_G15	LEDG[8] (STATUS)	PIN_F17
LEDR[7]	PIN_H19	LEDR[16]	PIN_G16		
LEDR[8]	PIN_J17	LEDR[17]	PIN_H15		

Displays (índice 0 = segmento a ... 6 = segmento g; acendem com 0)

Tabela 82 – pinagem (LEDs)

Display	Função	[0]	[1]	[2]	[3]	[4]	[5]	[6]
HEX7	dezena de A	AD17	AE17	AG17	AH17	AF17	AG18	AA14
HEX6	unidade de A	AA17	AB16	AA16	AB17	AB15	AA15	AC17
HEX5	dezena de B	AD18	AC18	AB18	AH19	AG19	AF18	AH18
HEX4	unidade de B	AB19	AA19	AG21	AH21	AE19	AF19	AE18
HEX2	sinal de F	AA25	AA26	Y25	W26	Y26	W27	W28
HEX1	dezena de F	M24	Y22	W21	W22	W25	U23	U24
HEX0	unidade de F	G18	F22	E17	L26	L25	J22	H22

5.4 Teste na placa

O projeto completo (src/Projeto1SD.qpf) foi compilado sem erros e gravado na DE2-115 por JTAG. Os casos abaixo foram conferidos na placa (A nas chaves SW17..SW13, B em SW12..SW8 e S em SW2..SW0):

Tabela 83 – teste na placa

S	A	B	Conta	HEX2 HEX1 HEX0	LEDG5..LEDG 0	LEDG8
000	00101 (+5)	10011 (-3)	$5 + (-3) = +2$	_ 0 2	000010	apagado
001	00010 (+2)	00101 (+5)	$2 - 5 = -3$	- 0 3	100011	apagado
001	11001 (-9)	01100 (+12)	$-9 - 12 = -21$	- 2 1	110101	apagado
000	11000 (-8)	11000 (-8)	$-8 + (-8) = -16$	- 1 6	110000	apagado
000	11111 (-15)	11111 (-15)	$-15 + (-15) = -30$	- 3 0	111110	apagado
000	01111 (+15)	01111 (+15)	$15 + 15 = +30$	_ 3 0	011110	apagado
001	00111 (+7)	00111 (+7)	$7 - 7 = 0$	_ 0 0	000000	apagado
000	10000 (-0)	00000 (+0)	$-0 + 0 = 0$	_ 0 0	000000	apagado
010	qualquer	10011 (-3)	$C2(-3) = 11101$	apagados	111101	apagado
010	qualquer	00101 (+5)	$C2(+5) = 00101$	apagados	000101	apagado
011	10000 (-0)	00000 (+0)	$-0 = +0$? sim	apagados	000000	aceso
011	00011 (+3)	10011 (-3)	$+3 = -3$? não	apagados	000000	apagado
100	00010 (+2)	10101 (-5)	$+2 > -5$? sim	apagados	000000	aceso
100	10010 (-2)	10101 (-5)	$-2 > -5$? sim	apagados	000000	aceso
101	10011 (-3)	10011 (-3)	$-3 < -3$? não	apagados	000000	apagado
101	10111 (-7)	00001 (+1)	$-7 < +1$? sim	apagados	000000	aceso
110	00101 (+5)	00011 (+3)	0101 AND 0011	apagados	000001	apagado
110	10101 (-5)	10011 (-3)	sinais 1 AND 1	apagados	100001	apagado
111	00101 (+5)	00011 (+3)	0101 XOR 0011	apagados	000110	apagado

Na coluna dos displays, "_" indica HEX2 apagado (resultado positivo).

[inserir figura: docs/relatorio/figuras/placa.jpg]

Figura 54 – ULA funcionando na DE2-115

6 Conclusão

O projeto atingiu todos os requisitos do enunciado: a ULA executa as oito operações sobre operandos de 5 bits em sinal-magnitude, entrega F em sinal-magnitude de 6 bits nos LEDs, indica as comparações no LED de STATUS, mostra |A|, |B| e |F| nos displays de 7 segmentos (os de F apenas na soma e na subtração) e foi testada com sucesso na placa DE2-115, com a pinagem configurada. Todo o circuito foi construído somente com portas lógicas.

A decisão que mais simplificou o projeto foi fazer as contas em complemento a 2 de 6 bits: um único somador resolve a soma e a subtração, sem risco de overflow, e a conversão SM $\leftrightarrow$ C2 fica concentrada em blocos pequenos (comp2 e c2_para_sm) que reaproveitam o mesmo atalho do complemento a 2. Nas comparações, tratar +0 e -0 como iguais exigiu atenção especial em comp2, comparador_igual e comp_maior, e usar o mesmo comp_maior com as entradas trocadas evitou um bloco a mais para “menor que”. Nos displays, os decodificadores diretos do binário para cada dígito dispensaram a conversão para BCD, ao custo de equações maiores, simplificadas por mapas de Karnaugh.

A divisão em blocos pequenos, cada um com tabela verdade e simulação próprias, foi o que permitiu achar e corrigir erros antes da integração. Os principais problemas encontrados foram de ligação, e não de lógica: barramentos com a ordem dos bits invertida, saídas com nomes repetidos, pinos sobrando no toplevel e instâncias duplicadas no esquemático. Organizar todos os blocos numa biblioteca única no repositório, com um projeto de teste para cada um e o fluxo de branches do Git, permitiu que os integrantes trabalhassem em paralelo sem copiar arquivos entre projetos.

Como melhorias, o somador em cascata (*ripple carry*) poderia ser trocado por um somador com antecipação de vai-um (*carry lookahead*), que reduz o atraso, e os decodificadores de display poderiam ser trocados por um conversor binário $\rightarrow$ BCD seguido de um decodificador BCD $\rightarrow$ 7 segmentos padrão, o mesmo bloco para todos os displays.